Index

1	Data Processing using Python Lib
2	Predicting House Prices using linear Regression
3	Predicting Stock Prices using linear Regression
4	Customer Churn Prediction using Linear regression
5	Spam Email Classification & Comparative Analysis
6	Credit Risk Assessment & Comparative Analysis
7	Anomaly Detection & Comparative Analysis
8	Student Performance Level Analysis & Comparative Analysis

1)Data processing using Python libararies

Using pandas to plot and clean data

Using matplotlib and seaborn to plt the outputs

1.Team Name Normalization

1 Some team names appear in different textual formats (e.g., case differences or extra spaces). Clean the team1 and team2 columns by:

2. Converting text to lowercase
3. Removing leading/trailing spaces
4. Verifying unique team names after cleaning

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df=pd.read_csv('/content/drive/MyDrive/data.csv')
print(df)
```

```
      id  season        city        date                  team1  \
0      1    2008   Bangalore  2008-04-18  Kolkata Knight Riders
1      2    2008  Chandigarh  2008-04-19    Chennai Super Kings
2      3    2008       Delhi  2008-04-19       Rajasthan Royals
3      4    2008      Mumbai  2008-04-20         Mumbai Indians
4      5    2008     Kolkata  2008-04-20        Deccan Chargers
..   ...     ...         ...         ...                    ...
572  573    2016      Raipur  2016-05-22       Delhi Daredevils
573  574    2016   Bangalore  2016-05-24          Gujarat Lions
574  575    2016       Delhi  2016-05-25    Sunrisers Hyderabad
575  576    2016       Delhi  2016-05-27          Gujarat Lions
576  577    2016   Bangalore  2016-05-29    Sunrisers Hyderabad

                          team2                 toss_winner toss_decision  \
0    Royal Challengers Bangalore  Royal Challengers Bangalore         field
1                Kings XI Punjab          Chennai Super Kings           bat
2               Delhi Daredevils             Rajasthan Royals           bat
3    Royal Challengers Bangalore               Mumbai Indians           bat
4          Kolkata Knight Riders              Deccan Chargers           bat
..                           ...                          ...           ...
572  Royal Challengers Bangalore  Royal Challengers Bangalore         field
573  Royal Challengers Bangalore  Royal Challengers Bangalore         field
574        Kolkata Knight Riders        Kolkata Knight Riders         field
575          Sunrisers Hyderabad          Sunrisers Hyderabad         field
576  Royal Challengers Bangalore          Sunrisers Hyderabad           bat

     result  dl_applied                      winner  win_by_runs  \
0    normal           0        Kolkata Knight Riders          140
1    normal           0          Chennai Super Kings           33
2    normal           0             Delhi Daredevils            0
3    normal           0  Royal Challengers Bangalore            0
4    normal           0        Kolkata Knight Riders            0
..      ...         ...                          ...          ...
572  normal           0  Royal Challengers Bangalore            0
573  normal           0  Royal Challengers Bangalore            0
574  normal           0          Sunrisers Hyderabad           22
575  normal           0          Sunrisers Hyderabad            0
576  normal           0          Sunrisers Hyderabad            8

     win_by_wickets player_of_match  \
0                 0     BB McCullum
1                 0      MEK Hussey
2                 9     MF Maharoof
3                 5      MV Boucher
4                 5       DJ Hussey
..              ...             ...
572               6         V Kohli
573               4  AB de Villiers
574               0    MC Henriques
575               4       DA Warner
576               0     BCJ Cutting

                                         venue          umpire1  \
0                          M Chinnaswamy Stadium        Asad Rauf
1          Punjab Cricket Association Stadium, Mohali        MR Benson
2                                 Feroz Shah Kotla        Aleem Dar
3                                 Wankhede Stadium         SJ Davis
4                                     Eden Gardens        BF Bowden
```

```
df.shape
```

```
(577, 18)
```

```
print(df['team1'], df['team2'])
```

```
0      Kolkata Knight Riders
1        Chennai Super Kings
2           Rajasthan Royals
3             Mumbai Indians
4            Deccan Chargers
               ...
572         Delhi Daredevils
573            Gujarat Lions
574      Sunrisers Hyderabad
575            Gujarat Lions
576      Sunrisers Hyderabad
Name: team1, Length: 577, dtype: object 0      Royal Challengers Bangalore
1                  Kings XI Punjab
2                 Delhi Daredevils
3      Royal Challengers Bangalore
4            Kolkata Knight Riders
                  ...
572    Royal Challengers Bangalore
573    Royal Challengers Bangalore
574          Kolkata Knight Riders
575            Sunrisers Hyderabad
576    Royal Challengers Bangalore
Name: team2, Length: 577, dtype: object
```

```
df['team1']=df['team1'].str.lower().str.strip() #.str.lower() to convert into lower case and space is removed
df['team2']=df['team2'].str.lower().str.lower()
print(df['team1'], df['team2'])
```

```
0      kolkata knight riders
1        chennai super kings
2           rajasthan royals
3             mumbai indians
4            deccan chargers
               ...
572         delhi daredevils
573            gujarat lions
574      sunrisers hyderabad
575            gujarat lions
576      sunrisers hyderabad
Name: team1, Length: 577, dtype: object 0      royal challengers bangalore
1                  kings xi punjab
2                 delhi daredevils
3      royal challengers bangalore
4            kolkata knight riders
                  ...
572    royal challengers bangalore
573    royal challengers bangalore
574          kolkata knight riders
575            sunrisers hyderabad
576    royal challengers bangalore
Name: team2, Length: 577, dtype: object
```

```
print(df['team1'].unique())
print(df['team2'].unique())
```

```
['kolkata knight riders' 'chennai super kings' 'rajasthan royals'
 'mumbai indians' 'deccan chargers' 'kings xi punjab'
 'royal challengers bangalore' 'delhi daredevils' 'kochi tuskers kerala'
 'pune warriors' 'sunrisers hyderabad' 'rising pune supergiants'
 'gujarat lions']
['royal challengers bangalore' 'kings xi punjab' 'delhi daredevils'
 'kolkata knight riders' 'rajasthan royals' 'mumbai indians'
 'chennai super kings' 'deccan chargers' 'pune warriors'
 'kochi tuskers kerala' 'sunrisers hyderabad' 'rising pune supergiants'
 'gujarat lions']
```

2.City Name Standardization

Standardize the city column by:

1)Replacing missing city names with "Unknown"

2)Converting all city names to title case

3)Counting matches played in each city

```
print(df['city'].isnull().sum())
```

```
7
```

```
df['city'].fillna('Unknown', inplace=True)
```

```
/tmp/ipython-input-357195196.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through cha
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col]


  df['city'].fillna('Unknown', inplace=True)
```

```
df['city'] = df['city'].str.title()
print(df['city'])
```

```
0       Bangalore
1      Chandigarh
2           Delhi
3          Mumbai
4         Kolkata
          ...
572        Raipur
573     Bangalore
574         Delhi
575         Delhi
576     Bangalore
Name: city, Length: 577, dtype: object
```

```
cmc=df['city'].value_counts()
print(cmc)
```

```
city
Mumbai            77
Bangalore         58
Kolkata           54
Delhi             53
Chennai           48
Chandigarh        42
Hyderabad         41
Jaipur            33
Pune              25
Durban            15
Centurion         12
Ahmedabad         12
Visakhapatnam     11
Dharamsala         9
Johannesburg       8
Unknown            7
Abu Dhabi          7
Cape Town          7
Port Elizabeth     7
Ranchi             7
Cuttack            7
Raipur             6
Sharjah            6
Rajkot             5
Kochi              5
Kimberley          3
East London        3
Nagpur             3
Bloemfontein       2
Indore             2
Kanpur             2
Name: count, dtype: int64
```

3. Toss Decision Text Analysis

Analyze the toss_decision column:

1)Extract unique decisions

2)Count how many times each decision was taken

3)Visualize the frequency using a bar chart

```
ud=df['toss_decision'].unique()
print(ud)
```

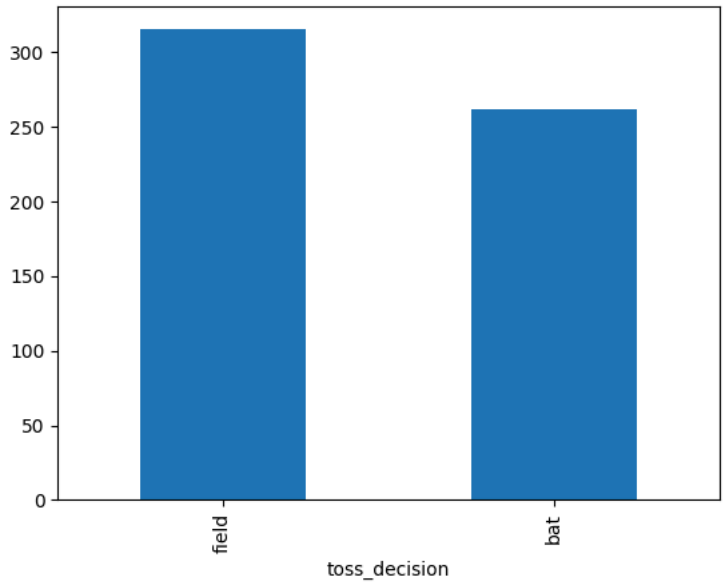
```
['field' 'bat']
```

```
dc=df['toss_decision'].value_counts()
print("Frequency of each toss decision:")
print(dc)
```

```
Frequency of each toss decision:
toss_decision
```

```
field    315
bat      262
Name: count, dtype: int64
```

```
bar=df['toss_decision'].value_counts().plot(kind='bar')
```



4, Winner Name Extraction

From the winner column:

1)Identify and remove rows where the match result was "No Result" or "Tie"

2)Count how many matches each team won after cleaning text values

```
df['result'].value_counts()
```

result	count
normal	568
tie	6
no result	3

dtype: int64

```
df_filtered = df[~df['result'].isin(['no result', 'tie'])]
print(df_filtered)
```

```
Original number of matches: 577
Number of matches after removing 'No Result' and 'Tie': 568
```

```
team_wins = df_filtered['winner'].value_counts()
print(team_wins)
```

```
winner
Mumbai Indians                 80
Chennai Super Kings            79
Royal Challengers Bangalore    69
Kolkata Knight Riders          68
Rajasthan Royals               61
Kings XI Punjab                61
Delhi Daredevils               56
Sunrisers Hyderabad            33
Deccan Chargers                29
Pune Warriors                  12
Gujarat Lions                   9
Kochi Tuskers Kerala            6
Rising Pune Supergiants         5
Name: count, dtype: int64
```

5. Player of the Match Text Frequency

Perform text analysis on player of the match:

1)Remove null values

2)Find the top 10 most frequent player names

3)Plot the results using a Seaborn bar plot

```
pn = df['player_of_match'].dropna()
print(pn)
```

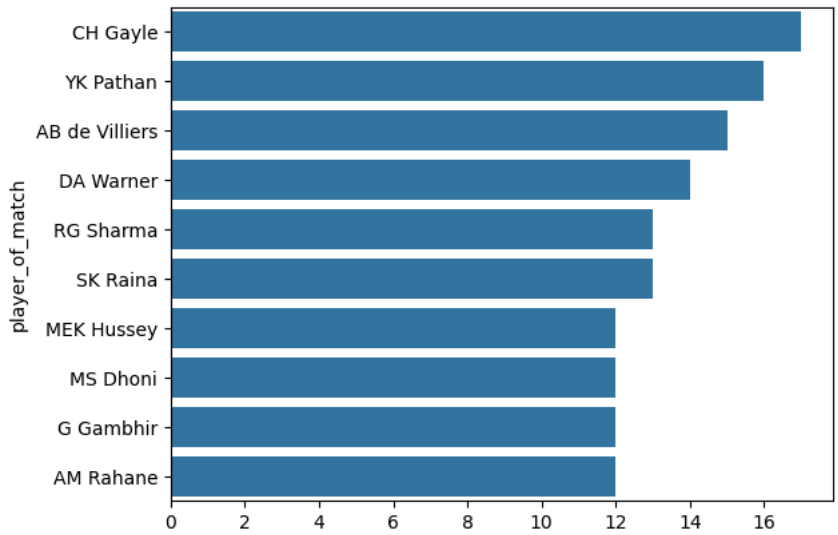
```
0         BB McCullum
1          MEK Hussey
2         MF Maharoof
3          MV Boucher
4           DJ Hussey
            ...
572           V Kohli
573    AB de Villiers
574      MC Henriques
575         DA Warner
576       BCJ Cutting
Name: player_of_match, Length: 574, dtype: object
```

```
tp=df['player_of_match'].value_counts().head(10)
print(tp)
```

```
player_of_match
CH Gayle          17
YK Pathan         16
AB de Villiers    15
DA Warner         14
RG Sharma         13
SK Raina          13
MEK Hussey        12
MS Dhoni          12
G Gambhir         12
AM Rahane         12
Name: count, dtype: int64
```

```
sns.barplot(y=tp.index, x=tp.values)
```

```
<Axes: ylabel='player_of_match'>
```



6. Venue Tokenization

Count how many matches were played in each venue and plot a bar chart for the top10.

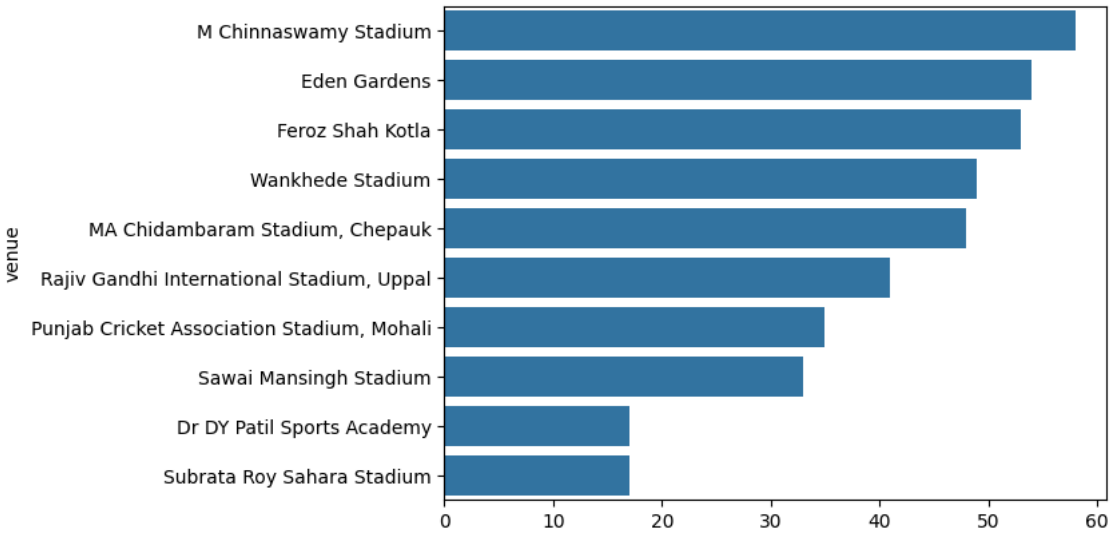
```
venue_counts = df['venue'].value_counts()
print(venue_counts.head(10))
```

```
venue
M Chinnaswamy Stadium                       58
Eden Gardens                                54
Feroz Shah Kotla                            53
```

```
Wankhede Stadium                            49
MA Chidambaram Stadium, Chepauk             48
Rajiv Gandhi International Stadium, Uppal   41
Punjab Cricket Association Stadium, Mohali  35
Sawai Mansingh Stadium                      33
Dr DY Patil Sports Academy                  17
Subrata Roy Sahara Stadium                  17
Name: count, dtype: int64
```

```
sns.barplot(y=venue_counts.head(10).index, x=venue_counts.head(10).values)
```

```
<Axes: ylabel='venue'>
```



7. Umpire Name Cleaning

Clean umpire columns (umpire1, umpire2, umpire3) by:

1)Replacing missing values with "Not Assigned"

2)Removing duplicate umpire names per match

3)Finding the most frequently officiating umpire

```
print(df['umpire1'].isnull().sum())
print(df['umpire2'].isnull().sum())
print(df['umpire3'].isnull().sum())
```

```
0
0
0
```

```
df['umpire3'].fillna('Not Assigned')
```

	umpire3
0	Not Assigned
1	Not Assigned
2	Not Assigned
3	Not Assigned
4	Not Assigned
...	...
572	Not Assigned
573	Not Assigned
574	Not Assigned
575	Not Assigned
576	Not Assigned

577 rows × 1 columns

dtype: object

```
print(df['umpire1'].isnull().sum())
print(df['umpire2'].isnull().sum())
print(df['umpire3'].isnull().sum())
```

```
0
0
0
```

```
umpire1=df['umpire1'].unique
umpire2=df['umpire2'].unique()
umpire3=df['umpire3'].unique()
```

```
print(df["umpire1"].head(10))
```

```
0    Asad Rauf
1    MR Benson
2    Aleem Dar
3     SJ Davis
4    BF Bowden
5    Aleem Dar
6    IL Howell
7    DJ Harper
8    Asad Rauf
9    Aleem Dar
Name: umpire1, dtype: object
```

8. Create a new text column match_summary by

combining:

team1, team2, winner, and season

Example: “MI vs CSK – MI won in 2019”

Display sample summaries.

```
pd.set_option('display.max_colwidth', None)
df['Match_Summary'] = df['team1'] + ' vs ' + df['team2'] + ' - ' + df['winner'].astype(str) + ' won in ' + df['season'].as
print(df['Match_Summary'].head())
```

```
0    kolkata knight riders vs royal challengers bangalore - Kolkata Knight Riders won in 2008
1                    chennai super kings vs kings xi punjab - Chennai Super Kings won in 2008
2                         rajasthan royals vs delhi daredevils - Delhi Daredevils won in 2008
3     mumbai indians vs royal challengers bangalore - Royal Challengers Bangalore won in 2008
4                deccan chargers vs kolkata knight riders - Kolkata Knight Riders won in 2008
Name: Match_Summary, dtype: object
```

9. Result Type Text Analysis

Analyze the result column:

Identify different textual result types

Count their occurrences

Visualize the distribution using a count plot

```
print(df['result'].unique())
```

```
['normal' 'tie' 'no result']
```

```
print(df['result'].value_counts())
```

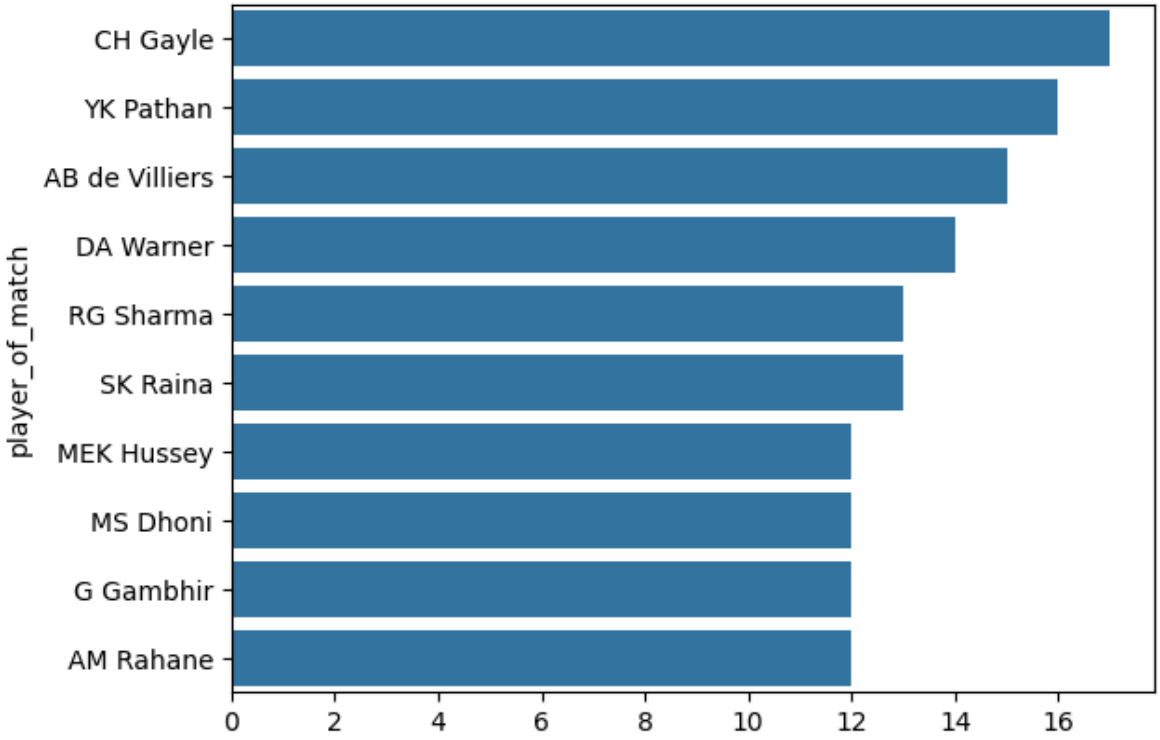
```
result
normal       568
tie            6
no result      3
Name: count, dtype: int64
```

```
sns.countplot(x=df['result'])
```

```
SK Raina          13
MEK Hussey        12
MS Dhoni          12
G Gambhir         12
AM Rahane         12
Name: count, dtype: int64
```

```
sns.barplot(y=tp.index, x=tp.values)
```

```
<Axes: ylabel='player_of_match'>
```



6. Venue Tokenization

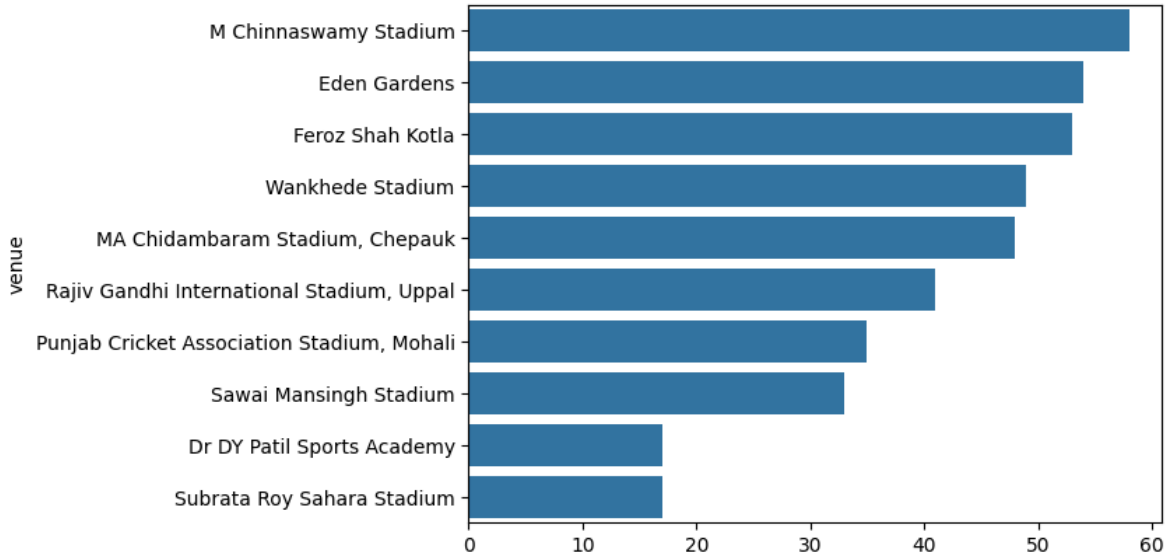
Count how many matches were played in each venue and plot a bar chart for the top10.

```
venue_counts = df['venue'].value_counts()
print(venue_counts.head(10))
```

```
venue
M Chinnaswamy Stadium                          58
Eden Gardens                                   54
Feroz Shah Kotla                               53
Wankhede Stadium                               49
MA Chidambaram Stadium, Chepauk                48
Rajiv Gandhi International Stadium, Uppal      41
Punjab Cricket Association Stadium, Mohali     35
Sawai Mansingh Stadium                         33
Dr DY Patil Sports Academy                     17
Subrata Roy Sahara Stadium                     17
Name: count, dtype: int64
```

```
sns.barplot(y=venue_counts.head(10).index, x=venue_counts.head(10).values)
```

```
<Axes: ylabel='venue'>
```



7. Umpire Name Cleaning

Clean umpire columns (umpire1, umpire2, umpire3) by:

1)Replacing missing values with "Not Assigned"

2)Removing duplicate umpire names per match

3)Finding the most frequently officiating umpire

```
print(df['umpire1'].isnull().sum())
print(df['umpire2'].isnull().sum())
print(df['umpire3'].isnull().sum())
```

```
0
0
0
```

```
df['umpire3'].fillna('Not Assigned')
```

	umpire3
0	Not Assigned
1	Not Assigned
2	Not Assigned
3	Not Assigned
4	Not Assigned
...	...
572	Not Assigned
573	Not Assigned
574	Not Assigned
575	Not Assigned
576	Not Assigned

577 rows × 1 columns

dtype: object

```
print(df['umpire1'].isnull().sum())
print(df['umpire2'].isnull().sum())
print(df['umpire3'].isnull().sum())
```

```
0
0
0
```

```
umpire1=df['umpire1'].unique
umpire2=df['umpire2'].unique()
umpire3=df['umpire3'].unique()
```

```
print(df["umpire1"].head(10))
```

```
0    Asad Rauf
1    MR Benson
2    Aleem Dar
3     SJ Davis
4    BF Bowden
5    Aleem Dar
6    IL Howell
7    DJ Harper
8    Asad Rauf
9    Aleem Dar
Name: umpire1, dtype: object
```

8. Create a new text column match_summary by

combining:

team1, team2, winner, and season

Example: “MI vs CSK – MI won in 2019”

Display sample summaries.

```
pd.set_option('display.max_colwidth', None)
df['Match_Summary'] = df['team1'] + ' vs ' + df['team2'] + ' - ' + df['winne
print(df['Match_Summary'].head())
```

```
0    kolkata knight riders vs royal challengers bangalore - Kolkata Knight Ri
1                     chennai super kings vs kings xi punjab - Chennai Super K
2                          rajasthan royals vs delhi daredevils - Delhi Darede
3     mumbai indians vs royal challengers bangalore - Royal Challengers Banga
4                deccan chargers vs kolkata knight riders - Kolkata Knight Ri
Name: Match_Summary, dtype: object
```

9. Result Type Text Analysis

Analyze the result column:

Identify different textual result types

Count their occurrences

Visualize the distribution using a count plot

```
print(df['result'].unique())
```

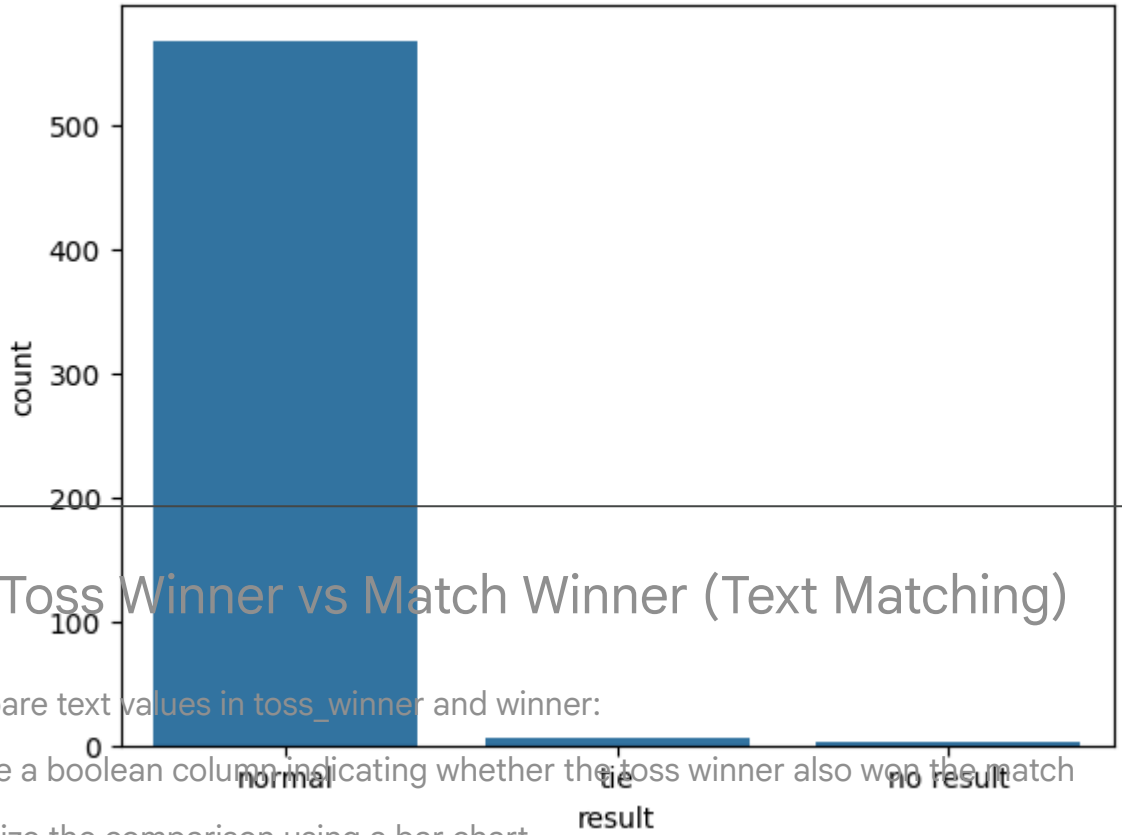
```
['normal' 'tie' 'no result']
```

```
print(df['result'].value_counts())
```

```
result
normal       568
tie            6
no result      3
Name: count, dtype: int64
```

```
sns.countplot(x=df['result'])
```

```
<Axes: xlabel='result', ylabel='count'>
```



10. Toss Winner vs Match Winner (Text Matching)

Compare text values in toss_winner and winner:

Create a boolean column indicating whether the toss winner also won the match

Visualize the comparison using a bar chart

```
df['toss_decision']=df['toss_winner'].str.strip().str.lower()
df['winner']=df['winner'].str.strip().str.lower()
df['toss_won']=df['toss_winner']==df['winner']
print(df['toss_won'].value_counts())
```

```
toss_won
False    577
Name: count, dtype: int64
```

```
sns.barplot(x=df['toss_won'].value_counts().index, y=df['toss_won'].value_cou
```

```
<Axes: xlabel='toss_won'>
```

EXE-2

Step-0 Data Processing

Python libraries such as pandas, matplotlib, and seaborn were used to handle and visualize the dataset. Pandas was used for loading, inspecting, and cleaning the data, while matplotlib and seaborn were used to generate plots that help analyze relationships between variables and understand patterns in the dataset.

Step-1 Problem Defination

The objective of this experiment is to build a machine learning model that can predict house prices using the given dataset. The dataset contains features such as area, number of bedrooms, location, and other property attributes. Since price is a continuous numeric value, this problem is a regression task. The model must learn how these features influence price and estimate it accurately.

Step-2 Data understanding

Using functions like df.head() and df.info(), the dataset structure was examined. Each row represents a house, and each column represents a feature describing that house.

The dataset contains:

Numerical features : Area, Bedrooms, Bathrooms, Age, Parking, Balcony, etc.

Categorical features : Location and Furnished

The target variable is Price, which is denoted as y, and the remaining features are used as input variables x. Since machine learning models require numeric input, categorical features must be encoded before training.

Step-3 Data Processing

Categorical variables such as Location and Furnished were converted into numerical format using one-hot encoding. This creates new columns such as:

- Location_Urban
- Location_Rural
- Furnished_Yes

This transformation was done using the get_dummies() function. Numerical features were already in usable format and therefore did not require modification. This preprocessing step ensures the dataset is suitable for machine learning algorithms. This aligns with the preprocessing stage of the pipeline where raw data is transformed into model-ready numerical form

Step-4 Data Selection and Model Training

As discussed earlier, Price was selected as the target variable (y), and all other columns were selected as input features (x). For demonstration purposes, the model was trained on the full dataset instead of splitting into training and testing sets.

The model chosen was Linear Regression, since the dependent variable is continuous. Regression models are specifically designed to predict numeric outputs, unlike classification models which predict categories.

Step 5-Prediction

After training, the model was used to generate predictions using the .predict() function. By providing input feature values (X), the model calculates estimated house prices based

on the learned relationship between features and target variable.

Step-6 Evaluation

The model performance was evaluated using:

Mean Squared Error (MSE) -- measures average squared difference between predicted and actual prices

R^2 Score - measures how well the model explains variation in price .

The obtained values showed MSE close to zero and R^2 close to 1, indicating that the model fit the training data extremely well. According to regression evaluation theory, lower MSE

Additionally, the results were visualized using a scatter plot to compare predicted prices with actual prices, allowing visual inspection of model accuracy.

Predicting Housing Prices: Develop a regression model to predict house prices based on features like location, size, and amenities. italicized text

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn import linear_model as lm
from sklearn.metrics import mean_squared_error, r2_score
```

```
df=pd.read_csv('/content/drive/MyDrive/Housing_Price_Dataset_120_Records.csv')
print(df.head())
```

```
  HouseID  Location  Area_sqft  Bedrooms  Bathrooms  Balcony  Parking  \
0    H001     Rural       1028         1          3        0        0
1    H002     Rural       2316         1          4        0        0
2    H003     Urban       1834         1          2        2        2
3    H004     Urban       1719         3          1        0        2
4    H005  Suburban       1118         2          3        0        0

   Age_Years Furnished   Price
0          5       Yes   55.84
1          3       Yes  103.88
2         23        No   86.42
3         14        No   96.77
4         13       Yes   63.94
```

Data Preprocessing

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 120 entries, 0 to 119
Data columns (total 10 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   HouseID    120 non-null    object
 1   Location   120 non-null    object
 2   Area_sqft  120 non-null    int64
 3   Bedrooms   120 non-null    int64
 4   Bathrooms  120 non-null    int64
 5   Balcony    120 non-null    int64
 6   Parking    120 non-null    int64
 7   Age_Years  120 non-null    int64
 8   Furnished  120 non-null    object
 9   Price      120 non-null    float64
dtypes: float64(1), int64(6), object(3)
memory usage: 9.5+ KB
```

```
pd.Index(['HouseID','Location','Area_sqft','Bedrooms','Bathrooms','Balcony','Parking','Age_Years','Furnished','Price'])
```

```
Index(['HouseID', 'Location', 'Area_sqft', 'Bedrooms', 'Bathrooms', 'Balcony',
       'Parking', 'Age_Years', 'Furnished', 'Price'],
      dtype='object')
```

```
df=pd.get_dummies(df,columns=['Furnished','Location'],drop_first=False)
```

```
df.head()
```

	HouseID	Area_sqft	Bedrooms	Bathrooms	Balcony	Parking	Age_Years	Price	Furnished_No	Furnished_Yes	Location_Rural
0	H001	1028	1	3	0	0	5	55.84	False	True	True
1	H002	2316	1	4	0	0	3	103.88	False	True	True
2	H003	1834	1	2	2	2	23	86.42	True	False	False
3	H004	1719	3	1	0	2	14	96.77	True	False	False
4	H005	1118	2	3	0	0	13	63.94	False	True	False

```
reg=lm.LinearRegression()
reg.fit(df[['Area_sqft','Bedrooms','Bathrooms','Balcony','Parking','Age_Years','Furnished_Yes','Location_Suburban','Locatio
```

```
▾ LinearRegression
LinearRegression()
```

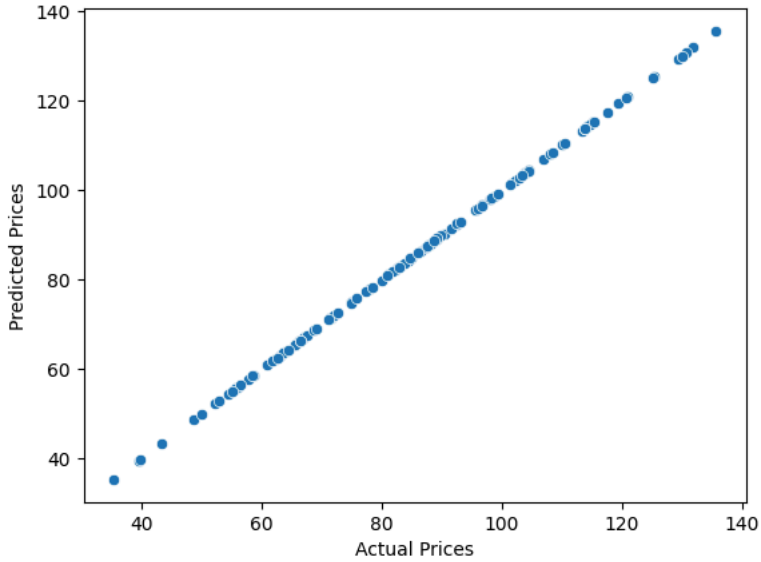
```
import pickle
with open('model_pickle','wb') as f:
  pickle.dump(reg,f)
```

```
y_pred = reg.predict(df[['Area_sqft','Bedrooms','Bathrooms','Balcony','Parking','Age_Years','Furnished_Yes','Location_Subur
mse = mean_squared_error(df.Price, y_pred)
r2 = r2_score(df.Price, y_pred)
print(f'Mean Squared Error: {mse}')
print(f'R^2 Score: {r2}')
```

```
Mean Squared Error: 6.344966032873642e-27
R^2 Score: 1.0
```

```
plot = sns.scatterplot(x=df.Price, y=y_pred)
plot.set_xlabel('Actual Prices')
plot.set_ylabel('Predicted Prices')
```

```
Text(0, 0.5, 'Predicted Prices')
```



EXE-3

Step-0 Libraries Used

Several Python libraries were used to perform data handling, model training, evaluation, and visualization for stock price prediction.

NumPy: was used for numerical operations and array manipulation required during model input preparation.

Pandas: was used for loading the dataset, filtering stock data, converting date formats, and creating new features such as the numerical time variable (Days).

Matplotlib and **Seaborn:** were used for visualization to plot stock price trends and analyze the distribution of values.

Scikit-learn: was used to implement machine learning functionality. The train_test_split module was used to divide data into training and testing sets, the LinearRegression model was used for prediction, and evaluation metrics such as **Mean Squared Error** and **R^2 score** were used to assess model performance.

Pickle: was used to save the trained model so it can be reused later without retraining.

Step-1 Problem Definition

The objective of this experiment is to develop a time series prediction model to forecast stock prices. The dataset contains historical stock data including Date, Open, High, Low, Close prices, and Volume for different companies. Since the output variable (stock price) is numeric and changes over time, this is a regression-based time series prediction problem.

Step-2 Data Understanding

The dataset was loaded using pandas and inspected to understand its structure. Each row represents a stock record for a specific date, and each column represents a feature describing that day's market performance.

The dataset contains the following attributes:

- Date → trading day
- Stock → company ticker symbol
- Open, High, Low, Close → price values
- Volume → number of shares traded

As shown in the dataset, the data contains multiple stocks such as AAPL and TSLA ,For modeling purposes, only one stock (AAPL) was selected so that a single time series could be analyzed.

Step-3 Data Preprocessing

The Date column was converted into datetime format to enable time-based calculations. A new feature called Days was created by subtracting the minimum date from each date value. This converts time into numerical form representing the number of days since the start of the dataset.

This step is necessary because machine learning models cannot directly process date values; they require numerical inputs.

Step-4 Feature Selection, Model Selection and Training

In this step, the input (X) was taken as the Days column, which represents time in numeric form, and the output (y) was selected as the closing stock price.

The dataset was divided into training and testing sets using an 80:20 ratio. Shuffling was disabled so that the chronological

order of dates remains unchanged. This is important for time-series data because predictions should always be made using past values to estimate future ones.

A Linear Regression model was chosen for this task because the target variable is continuous. The model was trained using the training data, where it learned the relationship between time and stock price by fitting a line that minimizes prediction error. After training, the model parameters were stored so that it could later be reused without retraining.

Step-6 Prediction

After training, the model was used to predict stock prices for the test dataset. The predictions generated represent estimated closing prices based solely on time progression.

As shown in the output, the predicted values form a continuous decreasing trend, indicating the model learned a linear relationship

Step-7 Evaluation

Model performance was evaluated using two regression metrics:

- Mean Squared Error (MSE):Measures average squared difference between predicted and actual values. Lower values indicate better performance.

- R^2 Score:Measures how well the model explains variance in the data. Values closer to 1 indicate better fit.

From the results :

MSE is approx.= 3072

R^2 is approx.= -11.21

The negative R^2 indicates the model performs poorly and does not fit the data well. This occurs because stock prices follow nonlinear patterns, while linear regression can only model straight-line relationships.

Predicting Stock Prices: Develop a time series prediction model to forecast stock prices.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn import linear_model as lm
from sklearn.metrics import r2_score,mean_squared_error
import pickle
```

```
df=pd.read_csv('/content/drive/MyDrive/stock_price_timeseries_dataset.csv')
df
```

	Date	Stock	Open	High	Low	Close	Volume
0	2020-01-01	AAPL	173.80	174.75	173.68	174.43	8204212
1	2020-01-02	AAPL	176.01	177.59	174.07	177.55	2766891
2	2020-01-03	AAPL	177.57	178.37	176.62	176.71	5721339
3	2020-01-06	AAPL	176.01	177.58	171.33	171.73	9242680
4	2020-01-07	AAPL	171.85	172.20	170.69	170.82	4416664
...	...	...	...	...	...	...	...
6520	2024-12-25	TSLA	225.09	225.21	221.66	223.18	9701165
6521	2024-12-26	TSLA	222.07	222.87	220.31	220.32	6045143
6522	2024-12-27	TSLA	219.94	221.69	218.38	219.57	8512677
6523	2024-12-30	TSLA	219.54	220.33	216.70	217.33	9137479
6524	2024-12-31	TSLA	216.47	219.61	216.19	218.41	2097124

6525 rows × 7 columns

Next steps: Generate code with df New interactive sheet

```
df_stock=df[df['Stock']=='AAPL']
df_stock.head()
```

	Date	Stock	Open	High	Low	Close	Volume
0	2020-01-01	AAPL	173.80	174.75	173.68	174.43	8204212
1	2020-01-02	AAPL	176.01	177.59	174.07	177.55	2766891
2	2020-01-03	AAPL	177.57	178.37	176.62	176.71	5721339
3	2020-01-06	AAPL	176.01	177.58	171.33	171.73	9242680
4	2020-01-07	AAPL	171.85	172.20	170.69	170.82	4416664

Next steps: Generate code with df_stock New interactive sheet

```
df_stock['Date']=pd.to_datetime(df_stock['Date'])
df_stock.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1305 entries, 0 to 1304
Data columns (total 7 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Date    1305 non-null   datetime64[ns]
 1   Stock   1305 non-null   object
 2   Open    1305 non-null   float64
 3   High    1305 non-null   float64
 4   Low     1305 non-null   float64
 5   Close   1305 non-null   float64
 6   Volume  1305 non-null   int64
dtypes: datetime64[ns](1), float64(4), int64(1), object(1)
memory usage: 113.9+ KB
```

```
/tmp/ipython-input-31740583.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
  df_stock['Date']=pd.to_datetime(df_stock['Date'])
```

```
df_stock['Days']=(df_stock['Date']-df_stock['Date'].min())
df_stock
```

```
/tmp/ipython-input-1482232864.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view
  df_stock['Days']=(df_stock['Date']-df_stock['Date'].min())
```

	Date	Stock	Open	High	Low	Close	Volume	Days
0	2020-01-01	AAPL	173.80	174.75	173.68	174.43	8204212	0 days
1	2020-01-02	AAPL	176.01	177.59	174.07	177.55	2766891	1 days
2	2020-01-03	AAPL	177.57	178.37	176.62	176.71	5721339	2 days
3	2020-01-06	AAPL	176.01	177.58	171.33	171.73	9242680	5 days
4	2020-01-07	AAPL	171.85	172.20	170.69	170.82	4416664	6 days
...	...	...	...	...	...	...	...	...
1300	2024-12-25	AAPL	207.96	210.35	206.18	208.42	4510463	1820 days
1301	2024-12-26	AAPL	208.69	214.63	207.23	213.99	4335705	1821 days
1302	2024-12-27	AAPL	213.99	217.12	212.77	216.24	2601207	1822 days
1303	2024-12-30	AAPL	216.62	217.17	213.72	215.53	7203690	1825 days
1304	2024-12-31	AAPL	215.26	218.50	215.06	217.00	5083372	1826 days

1305 rows × 8 columns

Next steps: Generate code with df_stock New interactive sheet

```
x=df_stock['Days']
y=df_stock['Close']
```

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,shuffle=False)
```

```
model=lm.LinearRegression()
model.fit(x_train.dt.days.values.reshape(-1,1), y_train)
with open('model_pickle','wb') as f:
  pickle.dump(model,f)
```

```
y_pred=model.predict(x_test.dt.days.values.reshape(-1,1))
y_pred
```

```
       150.49573228, 150.47262862, 150.40331765, 150.380214  ,
       150.35711034, 150.33400669, 150.31090303, 150.24159206,
       150.21848841, 150.19538475, 150.17228109, 150.14917744,
       150.07986647, 150.05676281, 150.03365916, 150.0105555 ,
       149.98745185, 149.91814088, 149.89503722, 149.87193357,
       149.84882991, 149.82572625, 149.75641529, 149.73331163,
       149.71020797, 149.68710432, 149.66400066, 149.59468969,
       149.57158604, 149.54848238, 149.52537873, 149.50227507,
       149.4329641 , 149.40986045, 149.38675679, 149.36365313,
       149.34054948, 149.27123851, 149.24813485, 149.2250312 ,
       149.20192754, 149.17882389, 149.10951292, 149.08640926,
       149.06330561, 149.04020195, 149.01709829, 148.94778733,
       148.92468367, 148.90158001, 148.87847636, 148.8553727 ,
       148.78606173, 148.76295808, 148.73985442, 148.71675077,
       148.69364711, 148.62433614, 148.60123249, 148.57812883,
       148.55502517, 148.53192152, 148.46261055, 148.43950689,
       148.41640324, 148.39329958, 148.37019593, 148.30088496,
       148.2777813 , 148.25467765, 148.23157399, 148.20847033,
       148.13915937, 148.11605571, 148.09295205, 148.0698484 ,
       148.04674474, 147.97743377, 147.95433012, 147.93122646,
```

```
       146.84353463, 146.82223097, 146.79914752, 146.77604566,
       146.75294001, 146.68362904, 146.66052538, 146.63742173,
       146.61431807, 146.59121441, 146.52190345, 146.49879979,
       146.47569613, 146.45259248, 146.42948882, 146.36017785,
       146.3370742 , 146.31397054, 146.29086689, 146.26776323,
       146.19845226, 146.17534861, 146.15224495, 146.12914129,
       146.10603764, 146.03672667, 146.01362301, 145.99051936,
       145.9674157 , 145.94431205, 145.87500108, 145.85189742,
       145.82879377, 145.80569011, 145.78258645, 145.71327549,
       145.69017183, 145.66706817, 145.64396452, 145.62086086,
       145.55154989, 145.52844624, 145.50534258, 145.48223893,
       145.45913527, 145.3898243 , 145.36672065, 145.34361699,
       145.32051333, 145.29740968, 145.22809871, 145.20499505,
       145.1818914 , 145.15878774, 145.13568409, 145.06637312,
       145.04326946, 145.02016581, 144.99706215, 144.97395849,
       144.90464753, 144.88154387, 144.85844021, 144.83533656,
       144.8122329 , 144.74292193, 144.71981828, 144.69671462,
       144.67361097, 144.65050731, 144.58119634, 144.55809269,
       144.53498903, 144.51188537, 144.48878172, 144.41947075,
       144.39636709, 144.37326344, 144.35015978, 144.32705613,
       144.25774516, 144.2346415 , 144.21153785, 144.18843419,
       144.16533053, 144.09601957, 144.07291591, 144.04981225,
       144.0267086 , 144.00360494, 143.93429397, 143.91119032,
       143.88808666, 143.86498301, 143.84187935, 143.77256838,
       143.74946473, 143.72636107, 143.70325741, 143.68015376,
       143.61084279, 143.58773913, 143.56463548, 143.54153182,
       143.51842817, 143.4491172 , 143.42601354, 143.40290989,
       143.37980623, 143.35670257, 143.28739161, 143.26428795,
       143.24118429, 143.21808064, 143.19497698, 143.12566601,
       143.10256236])
```

```
mse=mean_squared_error(y_test,y_pred)
mse
```

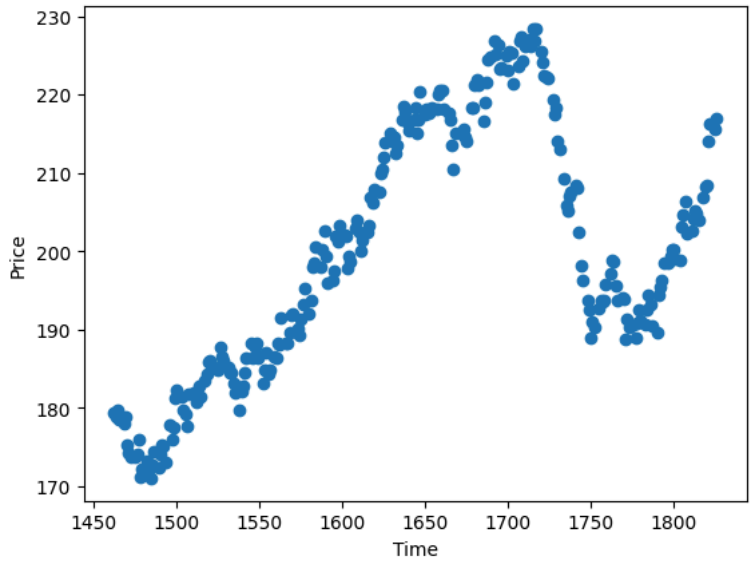
```
3072.378496877529
```

```
r2=r2_score(y_test,y_pred)
r2
```

```
-11.213999810715315
```

```
plt.scatter(x_test.dt.days,y_test)
plt.xlabel('Time')
plt.ylabel('Price')
```

```
Text(0, 0.5, 'Price')
```



EX-4

Step-0 Libraries Used

Pandas: is used to load and analyze data.

NumPy: is used for numerical operations.

Matplotlib and Seaborn: are used for plotting graphs.

Scikit-learn: is used for train test split, model training, prediction, and evaluation.

Pickle: is used to save the trained model for later use.

Step-1 Problem Definition

The goal is to predict whether a customer will churn or not using customer information such as age, subscription type, payment method, and charges. Since the output is either 0 or 1, this is a binary classification problem.

Step-2 Data Understanding

The dataset was loaded and checked using head() and info() to understand structure. Each row represents a customer and each column represents a feature.

The dataset contains numerical features such as age, tenure, charges, and support tickets, and categorical features such as gender, subscription type, payment method, and contract type. The target column is Churn.

Step-3 Data Processing

The CustomerID column was removed because it is only an identifier and does not help prediction. Categorical columns were converted into numerical form using one-hot encoding so that the model can understand them.

Step-4 Feature Selection and Model Training

All columns except Churn were selected as input (x), and Churn was selected as output (y). The dataset was split into training and testing sets in 80:20 ratio. A Logistic Regression model was chosen because it is suitable for binary classification. The model was trained on training data to learn patterns between customer features and churn.

Step-5 Prediction

After training, predictions were generated using the test data. The model outputs either 0 (not churn) or 1 (churn) based on learned patterns.

Step-6 Evaluation

The model performance was evaluated using accuracy, precision, recall, and F1 score. The accuracy was about 0.75, which shows the model performs reasonably well. However, precision and recall were lower, meaning the model has difficulty correctly identifying churn customers.

A confusion matrix was plotted to visualize prediction performance and to see correct and incorrect classifications.

Step-7 Feature Impact

A graph of model coefficients was plotted to see which features influence churn prediction the most. Features related to subscription type, payment method, and contract duration showed strong impact.

Customer Churn Prediction: Develop a model to predict customer churn in a subscription-based business.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score,confusion_matrix
import pickle
```

```
df=pd.read_csv('/content/drive/MyDrive/customer_churn_dataset.csv')
df.head()
```

	CustomerID	Gender	Age	Tenure_Months	SubscriptionType	MonthlyCharges	TotalCharges	PaymentMethod	SupportTickets	Co
0	CUST1000	Male	49	54	Standard	99.33	4912.77	Net Banking	1	
1	CUST1001	Female	56	8	Standard	12.01	604.59	Debit Card	2	
2	CUST1002	Male	66	27	Basic	57.62	2019.39	Cash	4	
3	CUST1003	Male	69	27	Premium	97.08	4848.88	Credit Card	7	
4	CUST1004	Male	49	34	Standard	54.69	4334.26	Credit Card	9	

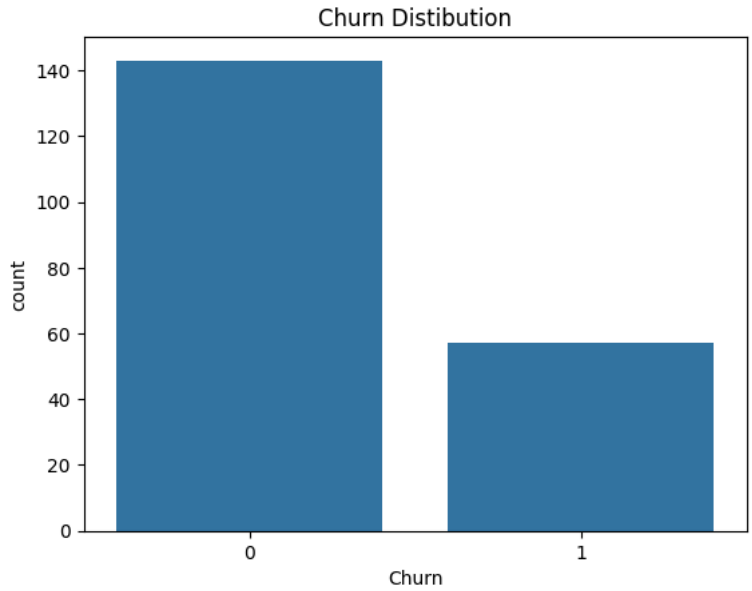
Next steps: Generate code with df | New interactive sheet

```
print("Shape:",df.shape)
df.info()
df.describe()
```

```
Shape: (200, 11)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 11 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   CustomerID        200 non-null    object
 1   Gender            200 non-null    object
 2   Age               200 non-null    int64
 3   Tenure_Months     200 non-null    int64
 4   SubscriptionType  200 non-null    object
 5   MonthlyCharges    200 non-null    float64
 6   TotalCharges      200 non-null    float64
 7   PaymentMethod     200 non-null    object
 8   SupportTickets    200 non-null    int64
 9   ContractType      200 non-null    object
 10  Churn             200 non-null    int64
dtypes: float64(2), int64(4), object(5)
memory usage: 17.3+ KB
```

	Age	Tenure_Months	MonthlyCharges	TotalCharges	SupportTickets	Churn
count	200.000000	200.000000	200.000000	200.000000	200.000000	200.000000
mean	44.405000	31.925000	51.348250	2536.105100	4.505000	0.285000
std	15.353816	16.390933	27.570371	1408.107184	2.967153	0.452547
min	18.000000	1.000000	6.380000	72.930000	0.000000	0.000000
25%	32.750000	21.000000	27.335000	1292.312500	2.000000	0.000000
50%	46.000000	32.000000	50.530000	2537.170000	4.000000	0.000000
75%	56.000000	46.250000	73.062500	3753.577500	7.000000	1.000000
max	69.000000	58.000000	99.970000	4984.530000	9.000000	1.000000

```
sns.countplot(x='Churn',data=df)
plt.title("Churn Distibution")
plt.show()
```




```
df=df.drop("CustomerID",axis=1)
df=pd.get_dummies(df,drop_first=True)
df.head()
```

	Age	Tenure_Months	MonthlyCharges	TotalCharges	SupportTickets	Churn	Gender_Male	SubscriptionType_Premium	Subscript
0	49	54	99.33	4912.77	1	0	True	False	
1	56	8	12.01	604.59	2	0	False	False	
2	66	27	57.62	2019.39	4	0	True	False	
3	69	27	97.08	4848.88	7	0	True	True	
4	49	34	54.69	4334.26	9	1	True	False	

Next steps: Generate code with df New interactive sheet

```
X=df.drop(["Churn"],axis=1)
y=df["Churn"]
X.head(),y.head()
```

```
(   Age  Tenure_Months  MonthlyCharges  TotalCharges  SupportTickets  \
 0   49            54           99.33       4912.77               1
 1   56             8           12.01        604.59               2
 2   66            27           57.62       2019.39               4
 3   69            27           97.08       4848.88               7
 4   49            34           54.69       4334.26               9

    Gender_Male  SubscriptionType_Premium  SubscriptionType_Standard  \
 0         True                     False                       True
 1        False                     False                       True
 2         True                     False                      False
 3         True                      True                      False
 4         True                     False                       True

    PaymentMethod_Credit Card  PaymentMethod_Debit Card  \
 0                      False                     False
 1                      False                      True
 2                      False                     False
 3                       True                     False
 4                       True                     False

    PaymentMethod_Net Banking  PaymentMethod_UPI  ContractType_One Year  \
 0                       True              False                  False
 1                      False              False                  False
 2                      False              False                  False
 3                      False              False                  False
 4                      False              False                   True

    ContractType_Two Year
 0                   True
 1                   True
 2                   True
 3                   True
 4                  False  ,
 0    0
```

```
1    0
2    0
3    0
4    1
Name: Churn, dtype: int64)
```

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=50)
print("Train size:",X_train.shape)
print("Test size:",X_test.shape)
```

```
Train size: (160, 14)
Test size: (40, 14)
```

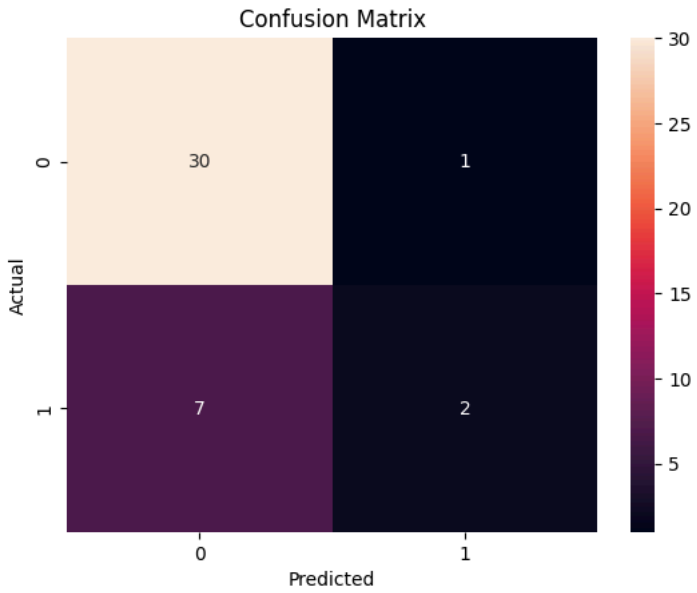
```
model=LogisticRegression(max_iter=1000)
model.fit(X_train,y_train)
with open("model","wb") as f:
    pickle.dump(model,f)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

```
y_pred=model.predict(X_test)
y_pred
```

```
array([0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0])
```

```
cm=confusion_matrix(y_test,y_pred)

sns.heatmap(cm,annot=True,fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```




```
print("Accuracy:",accuracy_score(y_test,y_pred))
print("Precision:",precision_score(y_test,y_pred))
print("Recall:",recall_score(y_test,y_pred))
print("F1 Score:",f1_score(y_test,y_pred))
```

```
Accuracy: 0.8
Precision: 0.6666666666666666
Recall: 0.2222222222222222
F1 Score: 0.3333333333333333
```

Ex-5

Step-0 Libraries Used

Pandas is used to load and handle the dataset.
NumPy helps with numerical operations.
Matplotlib and Seaborn are used to plot graphs and visualize model comparison and confusion matrix.
Scikit-learn is used for train-test splitting, TF-IDF text vectorization, model training, prediction, and evaluation.
TfidfVectorizer is used to convert text messages into numerical feature vectors using TF-IDF weighting so that machine learning algorithms can process the email text.
Classification algorithms such as Multinomial Naive Bayes, Logistic Regression, Support Vector Machine, Decision Tree, and Random Forest are used to train different models and compare their performance.
The metrics module from sklearn is used to calculate accuracy, precision, recall, F1 score, ROC-AUC score, and confusion matrix.
Pickle is used to save the trained model so it can be reused later without retraining.

Step-1 Problem Definition

The goal of this experiment is to build a system that can detect whether an email or SMS message is spam or not. This helps automatically filter unwanted messages and protect users from spam or fraudulent content. Since the output has only two values (spam or ham), this is a binary classification problem. The model must learn patterns in message text that distinguish spam messages from normal messages.

Step-2 Data Understanding

The dataset was loaded and checked using functions like head(), info(), and describe() to understand its structure. Each row represents one message and the columns represent its label and message text.

According to the dataset summary, it contains 5572 messages and 2 columns: Category and Message .

The Category column contains two values:

ham : normal message
spam : unwanted promotional or fraudulent message

The dataset is imbalanced because ham messages are more frequent than spam messages. For example, around 4825 messages are ham**,** which is much higher than spam messages. Understanding this distribution is important because imbalance can affect model evaluation.

Step-3 Data Processing

The labels were converted into numerical format by mapping:

ham : 0
spam : 1

This conversion is necessary because machine learning algorithms work only with numerical values.

After conversion, the message column was used as the input feature (X) and the Category column was used as the target variable (y). This prepares the dataset for machine learning training and prediction.

Step-4 Feature Extraction and Training

The dataset was split into training and testing sets using an 80:20 ratio so that the model trains on one part of the data and is evaluated on unseen data.

The message text was converted into numerical vectors using TF-IDF Vectorization (TfidfVectorizer) **.**TF-IDF stands for Term Frequency–Inverse Document Frequency. It measures how important a word is in a message relative to the entire dataset.

TF is calculated as:

TF(w,d) = (Number of times word w appears in document d) / (Total words in document)

IDF is calculated as:

IDF(w) = log(Total number of documents / Number of documents containing word w)

The final TF-IDF score is:

TF-IDF(w,d) = TF(w,d) × IDF(w)

This technique assigns higher weight to rare but informative words and reduces the importance of very common words.

After vectorization, the output matrices were:

Training matrix size : (4457, 7526)
Testing matrix size : (1115, 7526)

This means that 7526 unique words were extracted as features for classification.

Five machine learning models were trained:

Naive Bayes – a probabilistic classifier that works well for text data by calculating probabilities of words appearing in spam or ham messages.

Logistic Regression – a linear classifier that predicts the probability of a message being spam using the logistic function.

SVM – finds the optimal boundary that separates spam and non-spam messages in high-dimensional feature space.

Decision Tree – creates rule-based decisions by splitting features according to information gain.

Random Forest – an ensemble model that combines multiple decision trees to improve prediction accuracy and reduce overfitting.

Each model learns patterns in word usage that help classify messages as spam or ham.

Step-5 Prediction

After training the models, predictions were made on the test dataset. Each model classified the messages into either spam or ham. These predicted labels were then compared with the actual labels to measure model performance.

Step-6 Evaluation

The models were evaluated using multiple metrics:

Accuracy – measures the overall correctness of predictions.
Precision – measures how many predicted spam messages were actually spam.
Recall – measures how many actual spam messages were correctly detected.
F1 Score – harmonic mean of precision and recall, providing a balanced measure.

The results showed that SVM achieved the highest F1 score, making it the best performing model among all five algorithms.

Model comparison graphs were also plotted for Accuracy, Precision, and F1 Score to visually compare model performance. The graphs clearly show that SVM slightly outperforms the other models.

A confusion matrix was generated to analyze prediction results. The matrix shows:

955 ham messages correctly classified
142 spam messages correctly classified
18 spam messages incorrectly predicted as ham

This indicates that the model performs very well with very few misclassifications.

The ROC-AUC score was approximately 0.996, which indicates excellent classification performance and strong separation between spam and ham messages.

Step-7 Best Model Selection

The best model was selected based on the highest F1 score. In this experiment, Support Vector Machine (SVM) achieved the best performance.

The selected model was saved using pickle so that it can be reused later without retraining. Saving the model preserves its learned parameters and allows it to be loaded later for predictions in real applications.

Exp 5. Spam Email Detection: Build a spam email filter using text classification algorithms and perform comparative analysis. (Naïve Bayes, Logistic Regression, Support Vector Machine, Decision Tree, Random Forest)

Note: Use Evaluation Metrics (Accuracy, Precision, Recall, F1-Score for binary classification), (ROC-AUC, Accuracy, Precision, Recall, Weighted / Macro/ Micro Average F1-Score for Multiclassification task) and Confusion Matrix.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score,confusion_matrix,classif
import pickle
```

```
df=pd.read_csv('/content/drive/MyDrive/spam_email.csv')
df.head()
```

	Category	Message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Next steps: Generate code with df New interactive sheet

```
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 2 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Category  5572 non-null   object
 1   Message   5572 non-null   object
dtypes: object(2)
memory usage: 87.2+ KB
```

	Category	Message
count	5572	5572
unique	2	5157
top	ham	Sorry, I'll call later
freq	4825	30

```
df['Category']=df['Category'].map({'ham':0,'spam':1})
df.head()
```

	Category	Message
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

Next steps: Generate code with df New interactive sheet

```
X=df['Message']
y=df['Category']
```

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=0)
```

```
cv = TfidfVectorizer(stop_words='english')
X_train=cv.fit_transform(X_train)
X_test=cv.transform(X_test)
print("Training matrix shape:",X_train.shape)
print("Testing matrix shape:",X_test.shape)
```

```
Training matrix shape: (4457, 7526)
Testing matrix shape: (1115, 7526)
```

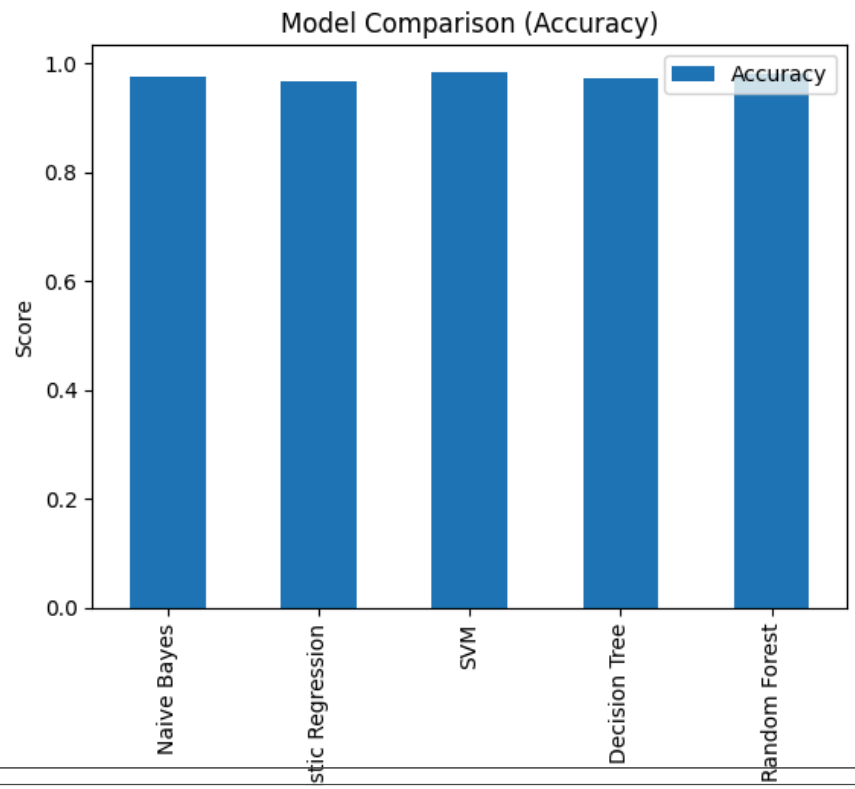
```
models={"Naive Bayes":MultinomialNB(),
        "Logistic Regression":LogisticRegression(max_iter=100),
        "SVM":SVC(),
        "Decision Tree":DecisionTreeClassifier(),
        "Random Forest":RandomForestClassifier()
        }
```

```
results=[]
for name,model in models.items():
    model.fit(X_train,y_train)
    pred=model.predict(X_test)
    acc=accuracy_score(y_test,pred)
    pre=precision_score(y_test,pred)
    rec=recall_score(y_test,pred)
    f1=f1_score(y_test,pred)
    results.append([name,acc,pre,rec,f1])
results_df=pd.DataFrame(results,columns=["Model","Accuracy","Precision","Recall","F1"])
results_df
```

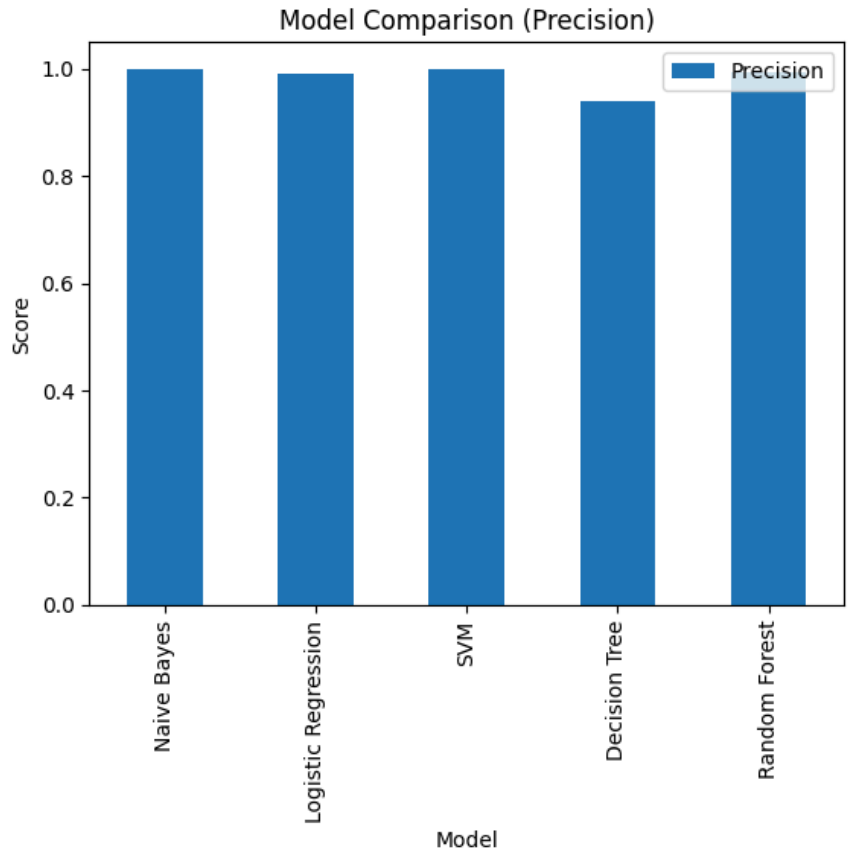
	Model	Accuracy	Precision	Recall	F1
0	Naive Bayes	0.975785	1.000000	0.83125	0.907850
1	Logistic Regression	0.965919	0.991935	0.76875	0.866197
2	SVM	0.983857	1.000000	0.88750	0.940397
3	Decision Tree	0.971300	0.938356	0.85625	0.895425
4	Random Forest	0.979372	0.992806	0.86250	0.923077

Next steps: Generate code with results_df New interactive sheet

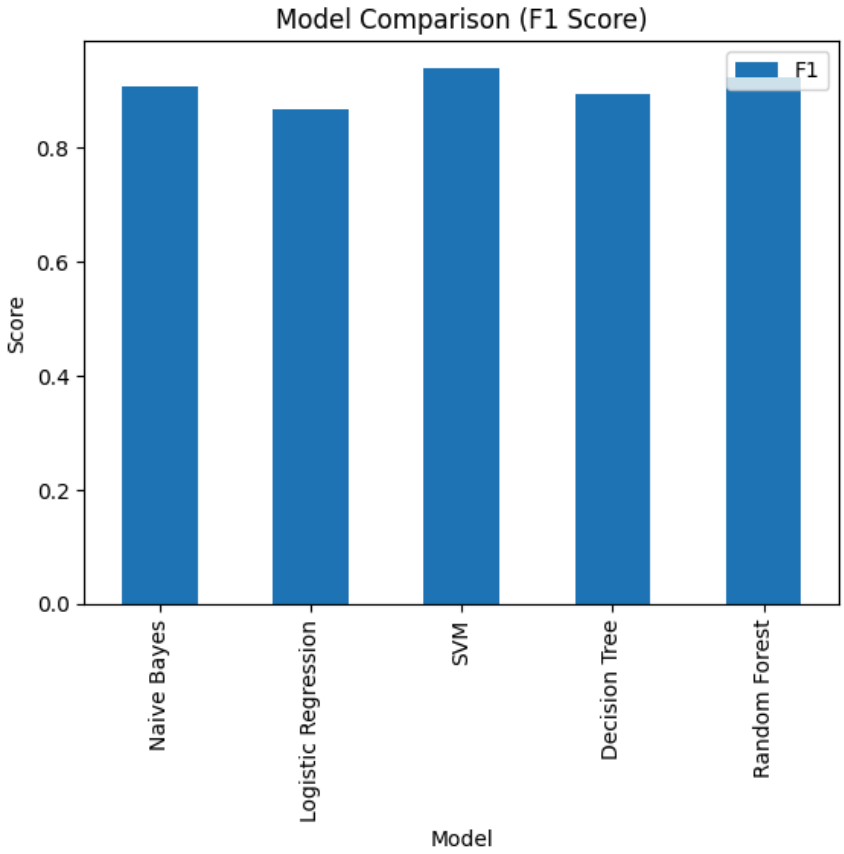
```
results_df.plot(x="Model",y="Accuracy",kind="bar")
plt.title("Model Comparison (Accuracy)")
plt.ylabel("Score")
plt.show()
```




```
results_df.plot(x="Model",y="Precision",kind="bar")
plt.title("Model Comparison (Precision)")
plt.ylabel("Score")
plt.show()
```




```
results_df.plot(x="Model",y="F1",kind="bar")
plt.title("Model Comparison (F1 Score)")
plt.ylabel("Score")
plt.show()
```



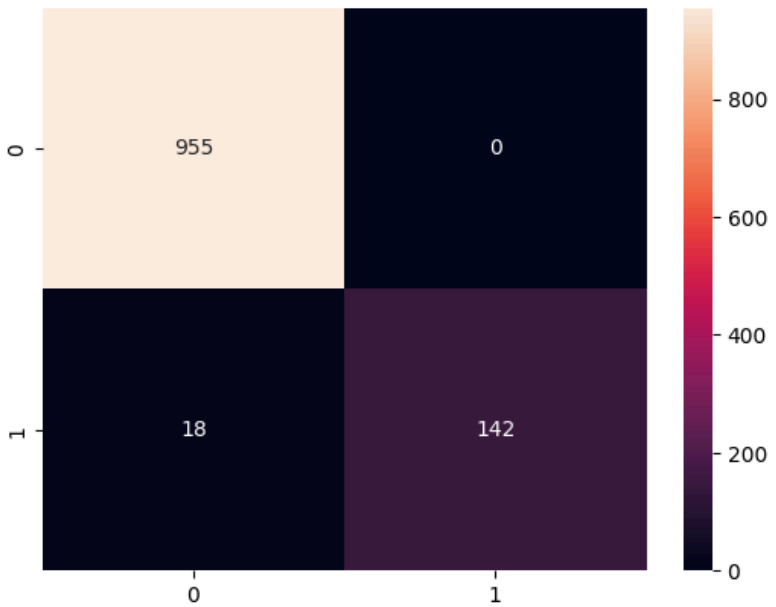

```
best_name=results_df.sort_values("F1",ascending=False).iloc[0,0]
best_model=models[best_name]
print("Best Model:",best_name)
```

```
Best Model: SVM
```

```
with open("spam_email_classifier.pkl","wb") as f:
    pickle.dump(best_model,f)
```

```
pred=best_model.predict(X_test)
cm=confusion_matrix(y_test,pred)
sns.heatmap(cm,annot=True,fmt="d")
```

```
<Axes: >
```



```
print("ROC AUC:",roc_auc_score(y_test,probs))
```

```
ROC AUC: 0.9963612565445027
```

```
print(classification_report(y_test,pred))
```

```
              precision    recall  f1-score   support

           0       0.98      1.00      0.99       955
           1       1.00      0.89      0.94       160

    accuracy                           0.98      1115
   macro avg       0.99      0.94      0.97      1115
weighted avg       0.98      0.98      0.98      1115
```

Ex-6

Step-0 Libraries Used

Pandas: is used to load and handle the dataset.
NumPy: helps in numerical operations.
Matplotlib and Seaborn: are used to visualize class distribution and model comparison graphs.
Scikit-learn: is used for train-test splitting, label encoding, model training, and evaluation. LogisticRegression, RandomForestClassifier, and XGBClassifier are used as classification models.
The metrics module: is used to calculate Accuracy, Precision, Recall, F1 (weighted, macro, micro), ROC-AUC, and Confusion Matrix.
Pickle: is used to save the trained best model for future use without retraining.

Step-1 Problem Definition

The objective of this experiment is to build a credit scoring model that classifies loan applicants into risk categories based on historical financial information.Since the output variable Credit_Risk contains multiple categories (High, Medium, Low), this is a multiclass classification problem.The goal is to compare different classification models and select the best-performing one.

Step-2 Data Understanding

The dataset contains 500 records and 11 columns
Important columns include:

- Age
- Annual_Income
- Employment_Years
- Credit_Score
- Loan_Amount
- Loan_Term_Months
- Existing_Loans_Count
- Debt_to_Income_Ratio
- Late_Payments_Last_2Yrs
- Credit_Risk (Target Variable)

The target column Credit_Risk is categorical (object type)
A countplot was generated to visualize the distribution of risk classes This helps understand class imbalance before training models.

Step-3 Data Processing

The target column Credit_Risk was converted into numerical format using LabelEncoder, because machine learning algorithms require numerical input.The Applicant_ID column was dropped since it is only an identifier and does not contribute to prediction.Features (X) were separated from the target (y).

Step-4 Train-Test Split and Model Training

The dataset was split using:
train_test_split(test_size=0.2, random_state=0)

This means:80% data used for training,20% data used for testing

Three models were trained:

- **Logistic Regression:**Linear model for classification.Performs well when classes are linearly separable.
- **Random Forest:**Ensemble of multiple decision trees.Reduces overfitting and improves stability.
- **XGBoost:**Gradient boosting algorithm. Builds trees sequentially to correct previous errors. Known for high performance in structured data.

Step-5 Model Evaluation and Comparative Analysis

Each model was evaluated using:

- Accuracy
- Precision (weighted)
- Recall (weighted)
- F1_weighted
- F1_macro
- F1_micro

Model Accuracy:

- Logistic Regression=0.68
- Random Forest=0.99
- XGBoost=1.00

XGBoost achieved perfect classification performance.

Step-6 Prediction

The confusion matrix shows:

- Class 0 = 70 correctly predicted
- Class 1 = 4 correctly predicted
- Class 2 = 26 correctly predicted

No misclassifications occurred.

The classification report confirms precision, recall, and F1-score of 1.00 for all classes
This indicates perfect separation of risk categories.

Step-7 Best Model Selection and Model Saving

The best model was selected based on highest F1_weighted score:
Best Model: XGBoost
The trained best model was saved using pickle:
pickle.dump(best_model,f)

Saving the model ensures:

- No need to retrain
- Reproducibility
- Deployment readiness

The saved model can later be loaded and used for predictions.

Exp 6: Credit Risk Assessment: Build a credit scoring model to assess the creditworthiness of applicants using historical financial data and perform comparative analysis (Logistic Regression, Random Forest, XGBoost)

Note: Use Evaluation Metrics (Accuracy, Precision, Recall, F1-Score for binary classification), (ROC-AUC, Accuracy, Precision, Recall, Weighted / Macro/ Micro Average F1-Score for Multiclassification task) and Confusion Matrix

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score,confusion_matrix,roc_auc_score,classificat
import pickle
```

```
df=pd.read_csv('/content/drive/MyDrive/credit_risk_assessment_500_samples.csv')
df.head()
```

	Applicant_ID	Age	Annual_Income	Employment_Years	Credit_Score	Loan_Amount	Loan_Term_Months	Existing_Loans_Count	De
0	1	59	153267	28	818	196649	12	2	
1	2	49	232745	0	436	175354	60	0	
2	3	35	974945	19	797	662297	48	3	
3	4	63	307164	29	776	831725	48	0	
4	5	28	685626	6	893	498625	60	3	

Next steps: Generate code with df New interactive sheet

```
print(df.shape)
df.info()
df.describe()
```

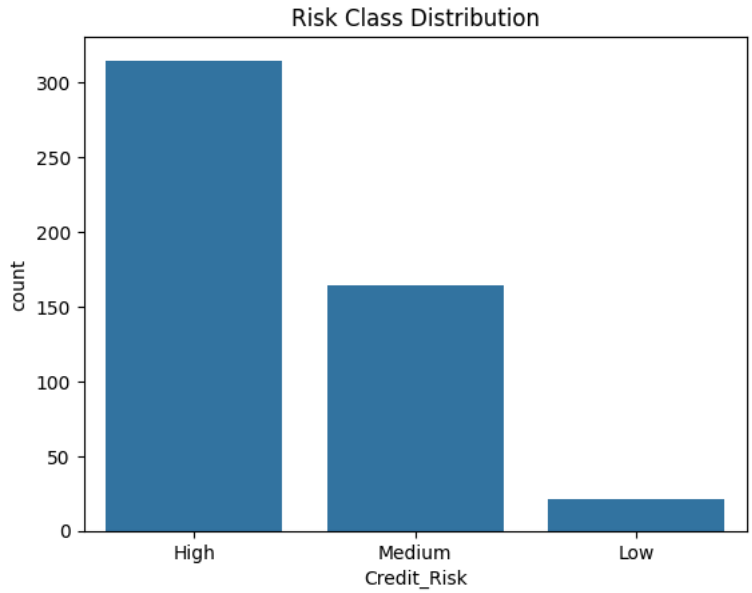
```
(500, 11)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Applicant_ID             500 non-null    int64
 1   Age                      500 non-null    int64
 2   Annual_Income            500 non-null    int64
 3   Employment_Years         500 non-null    int64
 4   Credit_Score             500 non-null    int64
 5   Loan_Amount              500 non-null    int64
 6   Loan_Term_Months         500 non-null    int64
 7   Existing_Loans_Count     500 non-null    int64
 8   Debt_to_Income_Ratio     500 non-null    float64
 9   Late_Payments_Last_2Yrs  500 non-null    int64
 10  Credit_Risk              500 non-null    object
dtypes: float64(1), int64(9), object(1)
memory usage: 43.1+ KB
```

1 to 8 of 8 entries Filter

index	Applicant_ID	Age	Annual_Income	Employment_Years	Credit_Score	Loan_Amount	Loan_Term_M
count	500.0	500.0	500.0	500.0	500.0	500.0	
mean	250.5	43.116	827131.654	14.702	596.394	521385.53	
std	144.4818327679989	12.733216939456186	384829.8617613202	8.713497925183917	171.33979518731104	281799.5141543726	17.117037182
min	1.0	21.0	150404.0	0.0	300.0	55901.0	
25%	125.75	32.0	506262.25	7.0	452.0	266621.75	
50%	250.5	44.0	833427.5	15.5	602.0	533640.0	
75%	375.25	53.0	1148353.0	22.0	732.25	765362.5	
max	500.0	64.0	1498464.0	29.0	898.0	999621.0	

Show 100 per page
Like what you see? Visit the data table notebook to learn more about interactive tables.

```
sns.countplot(x="Credit_Risk",data=df)
plt.title("Risk Class Distribution")
plt.show()
```




```
le=LabelEncoder()
df["Credit_Risk"]=le.fit_transform(df["Credit_Risk"])
```

```
df=df.drop("Applicant_ID",axis=1)
```

```
X=df.drop("Credit_Risk",axis=1)
y=df["Credit_Risk"]
```

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=0)
```

```
models={
    "Logistic Regression":LogisticRegression(max_iter=1000),
    "Random Forest":RandomForestClassifier(),
    "XGBoost":XGBClassifier(eval_metric="mlogloss")
    }
```

```
results=[]
for name,model in models.items():
    model.fit(X_train,y_train)
    pred=model.predict(X_test)
    acc=accuracy_score(y_test,pred)
    pre=precision_score(y_test,pred,average="weighted")
    rec=recall_score(y_test,pred,average="weighted")
    f1w=f1_score(y_test,pred,average="weighted")
    f1m=f1_score(y_test,pred,average="macro")
    f1mi=f1_score(y_test,pred,average="micro")
    results.append([name,acc,pre,rec,f1w,f1m,f1mi])
results_df=pd.DataFrame(results,columns=[
"Model","Accuracy","Precision","Recall",
"F1_weighted","F1_macro","F1_micro"])
results_df
```

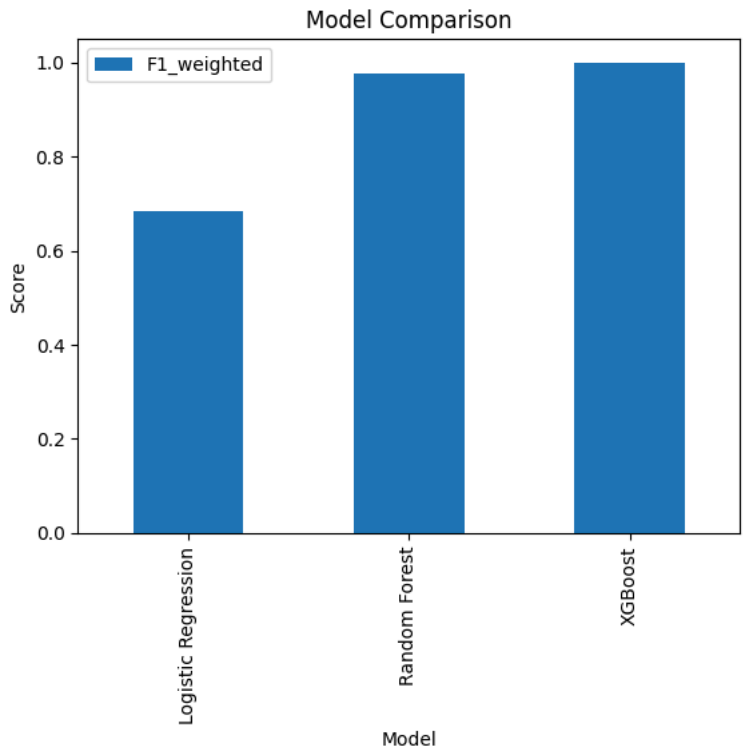
```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

	Model	Accuracy	Precision	Recall	F1_weighted	F1_macro	F1_micro
0	Logistic Regression	0.68	0.699529	0.68	0.683579	0.431938	0.68
1	Random Forest	0.99	0.990370	0.99	0.989380	0.946092	0.99
2	XGBoost	1.00	1.000000	1.00	1.000000	1.000000	1.00

Next steps: Generate code with results_df New interactive sheet

```
results_df.plot(x="Model",y="F1_weighted",kind="bar")
plt.title("Model Comparison")
```

```
plt.ylabel("Score")
plt.show()
```

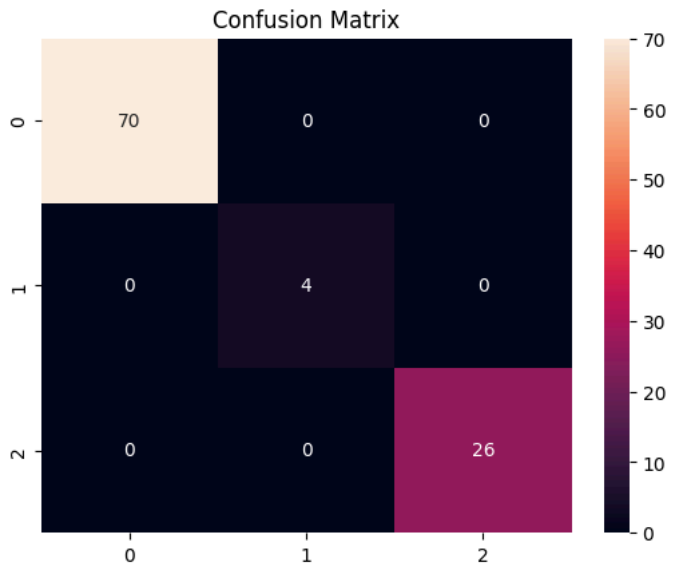



```
best_name=results_df.sort_values("F1_weighted",ascending=False).iloc[0,0]
best_model=models[best_name]
print("Best Model:",best_name)
```

```
Best Model: XGBoost
```

```
with open("best_model.pkl","wb") as f:
    pickle.dump(best_model,f)
```

```
pred=best_model.predict(X_test)
cm=confusion_matrix(y_test,pred)
sns.heatmap(cm,annot=True,fmt='d')
plt.title("Confusion Matrix")
plt.show()
```




```
print(classification_report(y_test,pred))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        70
           1       1.00      1.00      1.00         4
           2       1.00      1.00      1.00        26
```

```
    accuracy                           1.00       100
   macro avg       1.00      1.00      1.00       100
weighted avg       1.00      1.00      1.00       100
```

Exe-7

Step-0 Libraries Used

Pandas: is used to load and manage the dataset.
NumPy: is used for numerical operations and label conversion.
Matplotlib and Seaborn: are used to visualize data distribution, model comparison, and confusion matrix.
Scikit-learn: is used for scaling data, train-test splitting, model implementation, and evaluation metrics.
IsolationForest, LocalOutlierFactor, and OneClassSVM: are used as anomaly detection models.
Metrics: such as Accuracy, Precision, Recall, F1 Score, ROC-AUC, and Confusion Matrix are used to evaluate performance.
Pickle: is used to save the best-performing model for future reuse.

Step-1 Problem Definition

The objective of this experiment is to detect anomalies (outliers) in a dataset. In real world scenarios, anomaly detection is used in fraud detection, intrusion detection, and fault monitoring.

The dataset contains two numerical features and a target column Anomaly_Label, where:

- 0 = Normal data
- 1 = Anomaly

Since the output has two classes, this is a binary classification problem, but the models used are unsupervised anomaly detection algorithms.

Step-2 Data Understanding

The dataset contains 500 rows and 3 columns

- Feature_1
- Feature_2
- Anomaly_Label

The dataset is imbalanced:

- Most records are normal (0)
- Very few records are anomalies (1)

confirms that anomalies are rare, which is typical in anomaly detection problems.
This imbalance makes accuracy alone unreliable, so precision, recall, and F1-score become important.

Step-3 Data Preprocessing

The features were separated:

- X = df[["Feature_1","Feature_2"]]
- y = df["Anomaly_Label"]

Since anomaly detection models are distance-based or boundary-based, feature scaling is necessary.
StandardScaler was used to normalize data so that both features contribute equally to distance calculations.

After scaling, the dataset was split:

- 80% = Training
- 20% = Testing

Step-4 Model Training

Three anomaly detection models were implemented:

1. Isolation Forest

Works by randomly selecting features and splitting values.
Anomalies are easier to isolate and require fewer splits.
Uses contamination parameter to define expected anomaly proportion.

2. Local Outlier Factor (LOF)

Detects anomalies by comparing local density of a point to its neighbors.
Points with significantly lower density are marked as anomalies.

3. One-Class SVM

Creates a boundary around normal data points.
Points outside the boundary are considered anomalies.

Since these are unsupervised models, they were trained using only feature data (X_train).

Predictions returned values:

- -1 = anomaly
- 1 = normal

These were converted to:

-1 = 1 (anomaly)
1 = 0 (normal)

to match the dataset labels.

Step-5 Model Evaluation and Comparative Analysis

The models were evaluated using:

- Accuracy
- Precision
- Recall
- F1 Score
- ROC-AUC

Model	F1 Score
Isolation Forest	0.727
Local Outlier Factor	0.000
One-Class SVM	0.316

Isolation Forest performed best overall.

Interpretation:

- Isolation Forest achieved high recall (1.0), meaning it detected all anomalies.
- LOF failed to detect anomalies properly.
- One-Class SVM detected some anomalies but with lower performance.

clearly shows Isolation Forest having the highest F1 score.

Step-6 Confusion Matrix Analysis

The confusion matrix for Isolation Forest shows:

Actual	Predicted Normal	Predicted Anomaly
Normal	86	6
Anomaly	0	8

Interpretation:

- 86 normal points correctly identified
- 8 anomalies correctly detected
- 6 normal points incorrectly classified as anomalies (False Positives)
- 0 anomalies missed (False Negatives)

This means the model successfully detected all anomalies but slightly over-flagged some normal points.

In anomaly detection, missing anomalies (false negatives) is more critical than false positives, so this result is acceptable.

Step-7 Best Model Selection and Model Saving

The best model was selected based on highest F1 score:
Best Model: Isolation Forest

The trained model was saved using pickle:
pickle.dump(best_model,f)

Saving the model ensures:

- No need to retrain
- Faster deployment
- Reproducibility

The saved model can later be loaded and used for anomaly detection on new data.

Exp 7: Anomaly Detection: Implement an anomaly detection system for detecting outliers in data (e.g., fraud detection) and perform comparative analysis. (Isolation Forest, Local Outlier Factor, One-Class SVM)

Note: Use Evaluation Metrics (Accuracy, Precision, Recall, F1-Score for binary classification), (ROC-AUC, Accuracy, Precision, Recall, Weighted / Macro/ Micro Average F1-Score for Multiclassification task) and Confusion Matrix.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.svm import OneClassSVM
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score,confusion_matrix,roc_au
import pickle
```

```
df=pd.read_csv('/content/drive/MyDrive/anomaly_detection_dataset_500_samples.csv')
df.head()
```

	Feature_1	Feature_2	Anomaly_Label
0	43.259073	53.716320	0
1	43.815246	43.397717	0
2	46.942411	42.966695	0
3	48.911594	55.493884	0
4	52.575238	69.263657	0

Next steps: Generate code with df New interactive sheet

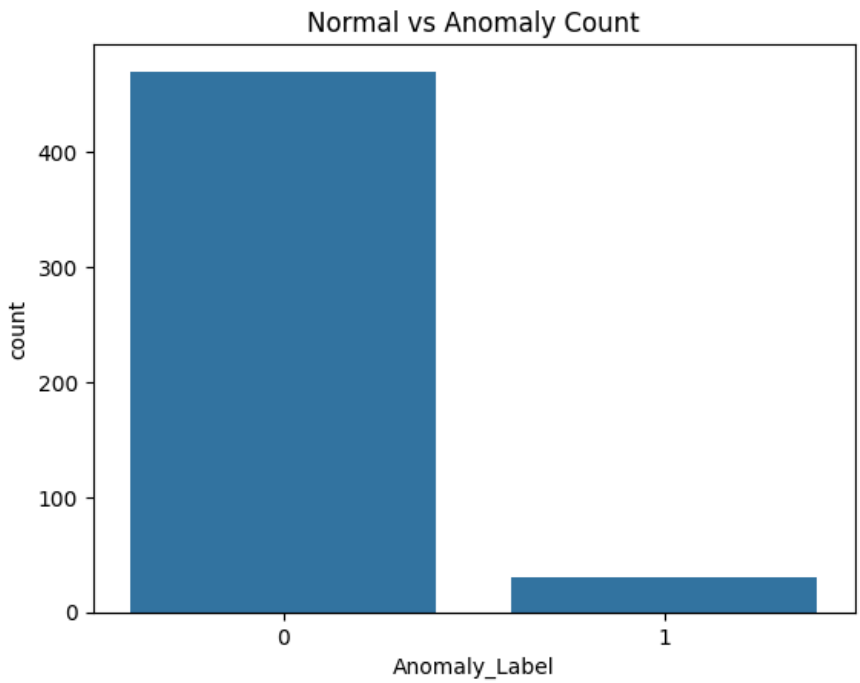
```
print(df.shape)
df.info()
df.describe()
```

```
(500, 3)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 3 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Feature_1      500 non-null    float64
 1   Feature_2      500 non-null    float64
 2   Anomaly_Label  500 non-null    int64
dtypes: float64(2), int64(1)
memory usage: 11.8 KB
```

	Feature_1	Feature_2	Anomaly_Label
count	500.000000	500.000000	500.000000
mean	52.422862	52.573850	0.060000
std	10.600455	10.641964	0.237725
min	33.793663	38.490394	0.000000
25%	47.027724	47.097147	0.000000
50%	50.447020	50.566493	0.000000
75%	53.827497	54.163940	0.000000
max	95.539911	95.402821	1.000000

```
sns.countplot(x="Anomaly_Label",data=df)
plt.title("Normal vs Anomaly Count")
```

```
plt.show()
```




```
X=df[["Feature_1","Feature_2"]]
y=df["Anomaly_Label"]
```

```
sc=StandardScaler()
X_scaled=sc.fit_transform(X)
```

```
X_train,X_test,y_train,y_test=train_test_split(X_scaled,y,test_size=0.2,random_state=0)
```

```
models={
"Isolation Forest":IsolationForest(contamination=0.1,random_state=0),
"Local Outlier Factor":LocalOutlierFactor(novelty=True),
"One-Class SVM":OneClassSVM(nu=0.1)
}
```

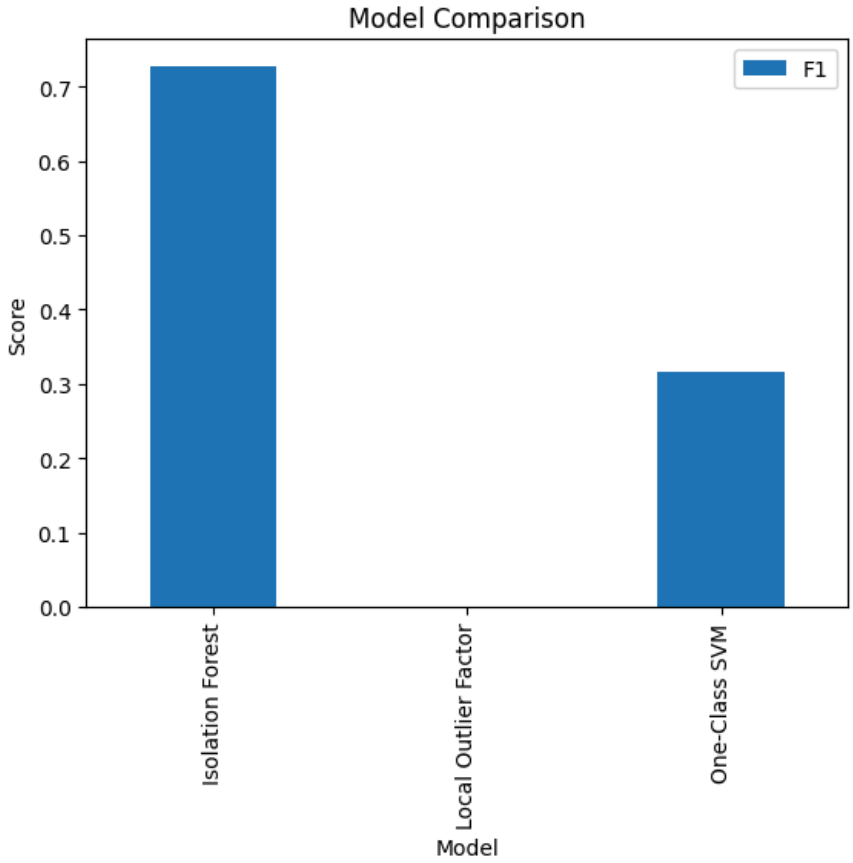
```
results=[]
for name,model in models.items():
    model.fit(X_train)
    pred=model.predict(X_test)
    pred=np.where(pred==-1,1,0)
    acc=accuracy_score(y_test,pred)
    pre=precision_score(y_test,pred)
    rec=recall_score(y_test,pred)
    f1=f1_score(y_test,pred)
    auc=roc_auc_score(y_test,pred)
    results.append([name,acc,pre,rec,f1,auc])
results_df=pd.DataFrame(results,columns=["Model","Accuracy","Precision","Recall","F1","ROC_AUC"])
results_df
```

	Model	Accuracy	Precision	Recall	F1	ROC_AUC
0	Isolation Forest	0.94	0.571429	1.000	0.727273	0.967391
1	Local Outlier Factor	0.88	0.000000	0.000	0.000000	0.478261
2	One-Class SVM	0.87	0.272727	0.375	0.315789	0.644022

Next steps: Generate code with results_df New interactive sheet

```
results_df.plot(x="Model",y="F1",kind="bar")
plt.title("Model Comparison")
```

```
plt.ylabel("Score")
plt.show()
```

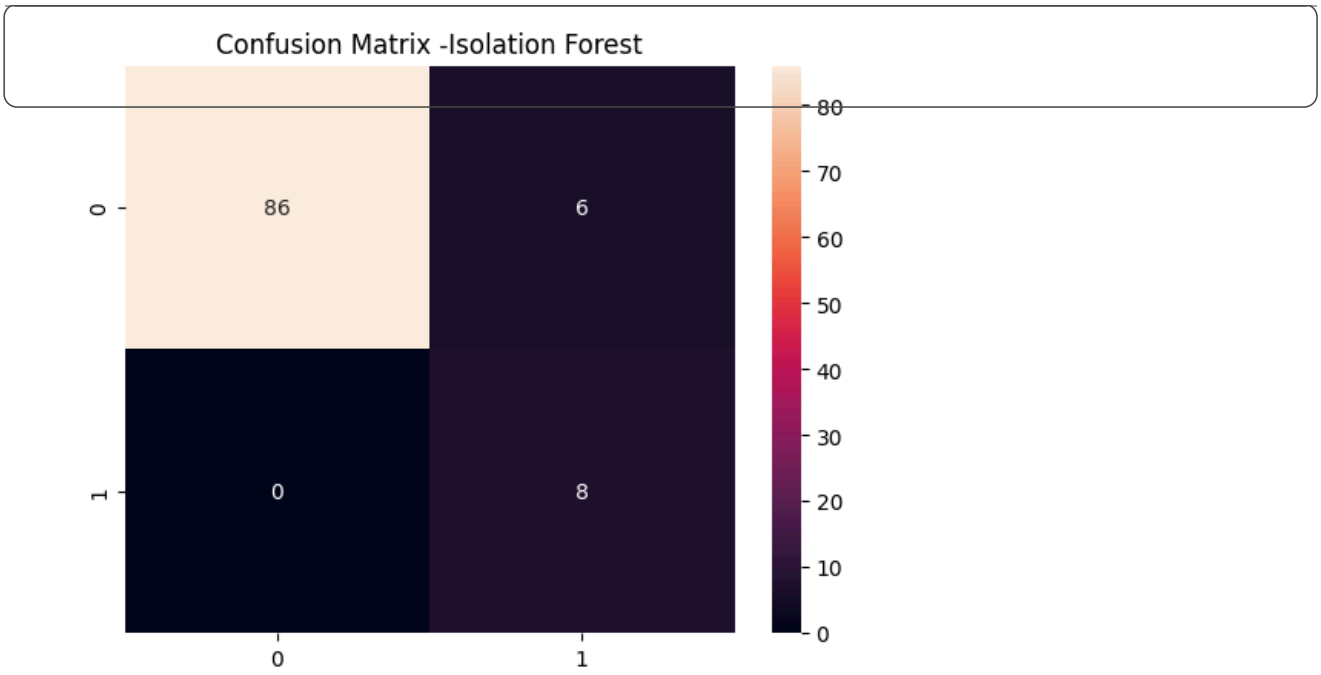



```
best=results_df.sort_values("F1",ascending=False).iloc[0,0]
best_model=models[best]
print("Best Model:",best)
```

```
Best Model: Isolation Forest
```

```
with(open("best_model.pkl","wb")) as f:
    pickle.dump(best_model,f)
```

```
pred=best_model.predict(X_test)
pred=np.where(pred==-1,1,0)
cm=confusion_matrix(y_test,pred)
sns.heatmap(cm,annot=True,fmt='d')
plt.title("Confusion Matrix -"+best)
plt.show()
```


Confusion Matrix -Isolation Forest
86
6
0
8
0
1
0
1
80
70
60
50
40
30
20
10
0

Exe-8

Step-0 Libraries Used

- **Pandas:** is used to load and manage the dataset.
- **NumPy:** is used for numerical operations.
- **Matplotlib and Seaborn:** are used to visualize class distribution and model comparison.
- **Scikit-learn:** is used for data preprocessing, scaling, train-test splitting, model implementation, and evaluation.
- **StandardScaler:** is used to normalize feature values.
- **RandomForestClassifier, DecisionTreeClassifier, LogisticRegression, KNeighborsClassifier, and XGBClassifier**: are used as classification models.
- **Metrics:** such as Accuracy, Precision, Recall, F1 (weighted, macro, micro), Confusion Matrix, and Classification Report are used for performance evaluation.
- **Pickle:** is used to save the best model for reuse without retraining.

Step-1 Problem Definition

The objective of this experiment is to classify students into different performance levels based on academic and behavioral features.The target variable Performance_Level contains three classes (0, 1, 2), making this a multiclass classification problem.The goal is to compare multiple machine learning models and determine which model performs best.

Step-2 Data Understanding

The dataset contains 500 rows and 8 columns

Features include:

- Study_Hours
- Attendance_Percentage
- Assignment_Score
- Internal_Marks
- Participation (categorical)
- Internet_Access (categorical)
- Previous_Grade (categorical)
- Performance_Level (target)

The countplot indicates that classes 0 and 1 dominate the dataset, while class 2 has very few samples. This imbalance explains the warning messages seen during metric calculation.

Step-3 Data Preprocessing

Since some columns are categorical (Participation, Internet_Access, Previous_Grade), they were converted into numerical format using:

pd.get_dummies(drop_first=True)

This creates binary indicator columns for categorical values.

After encoding:

- X = all features except Performance_Level
- y = Performance_Level

StandardScaler was used to normalize the features. Scaling ensures that models like Logistic Regression and KNN perform properly since they are sensitive to feature magnitude.

The dataset was then split:

- 80% = Training
- 20% = Testing

Step-4 Model Training

Five classification models were implemented:

1. Random Forest

Ensemble of multiple decision trees.
Reduces overfitting and improves stability.

2. Decision Tree

Creates rule-based splits based on feature importance.

3. Logistic Regression (Multinomial)

Linear classifier using probability-based prediction.
Configured with higher max_iter to ensure convergence.

4. K-Nearest Neighbors (KNN)

Classifies data based on nearest neighbors using distance metrics.

5. XGBoost

Boosting algorithm that builds trees sequentially to reduce errors.
Performs well in structured datasets.

Step-5 Model Evaluation and Comparative Analysis

Each model was evaluated using:

- Accuracy
- Precision (weighted)
- Recall (weighted)
- F1_weighted
- F1_macro
- F1_micro

From the results table

Model	**Accuracy**
Random Forest	0.93
Decision Tree	0.93
Logistic Regression	0.98
KNN	0.73
XGBoost	0.95

Logistic Regression achieved the highest weighted F1 score (0.975)

The bar graph clearly shows Logistic Regression performing best among all models.

Important Observation

Macro F1 scores are lower compared to weighted F1 scores. This is because:

- Class 2 has very few samples
- Models struggle to predict minority class

This is also confirmed by warning messages in the output

Step-6 Confusion Matrix Analysis

The confusion matrix shows:

Actual	Predicted 0	Predicted 1	Predicted 2
0	43	0	0
1	1	55	0
2	0	1	0

Interpretation:

- Classes 0 and 1 are predicted very accurately.
- Class 2 (minority class) is not predicted correctly.
- Only 1 sample belongs to class 2 in test set, and it was misclassified.

This explains why macro average is lower than weighted average.

Step-7 Best Model Selection and Saving

The best model was selected based on highest F1_weighted score:
Best Model: Logistic Regression

The model was saved using pickle:
pickle.dump(best_model,f)

Saving ensures:

- Reproducibility
- Faster deployment
- No retraining required

The saved model can later be loaded and used for prediction.

Exp 8: Implement Multiclass Classification models for students Performance Level analysis and perform comparative analysis. (Random Forest, Decision Tree, Multinomial Logistic Regression, XGBoost, K-Nearest Neighbors).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder,StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score,confusion_matrix,classif
import pickle
```

```
df=pd.read_csv('/content/drive/MyDrive/multiclass_classification_dataset_500_samples.csv')
df.head()
```

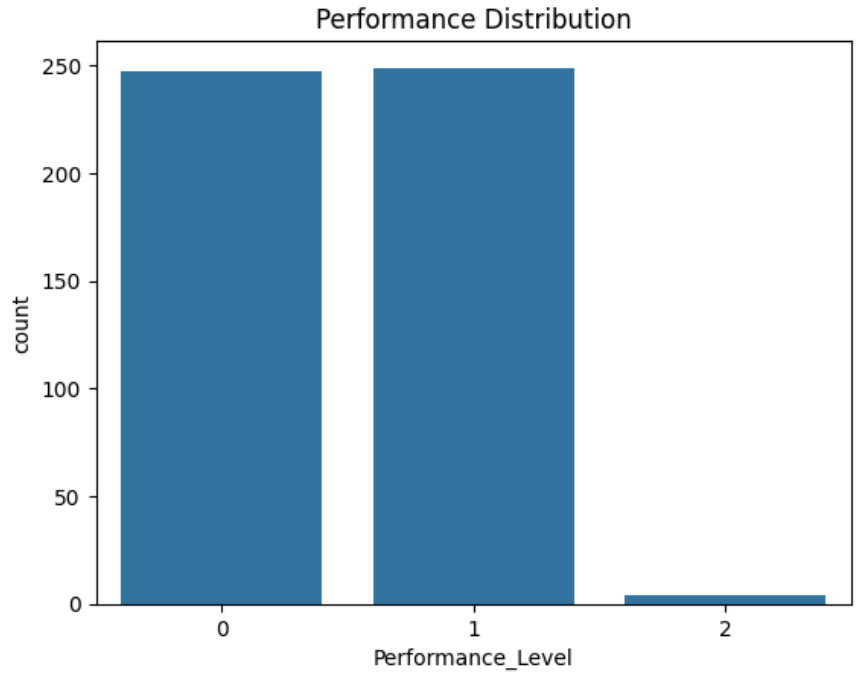
	Study_Hours	Attendance_Percentage	Assignment_Score	Internal_Marks	Participation	Internet_Access	Pr
0	22.48	84.26	86.79	76.68	Medium	Yes	
1	19.31	94.09	81.10	56.73	Low	Yes	
2	23.24	61.01	70.72	52.73	Medium	Yes	
3	27.62	80.63	62.24	64.95	Low	Yes	
4	18.83	68.49	78.38	62.45	High	Yes	

Next steps: Generate code with df New interactive sheet

```
print(df.shape)
df.describe()
df.info()
```

```
(500, 8)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 8 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Study_Hours            500 non-null    float64
 1   Attendance_Percentage  500 non-null    float64
 2   Assignment_Score       500 non-null    float64
 3   Internal_Marks         500 non-null    float64
 4   Participation          500 non-null    object
 5   Internet_Access        500 non-null    object
 6   Previous_Grade         500 non-null    object
 7   Performance_Level      500 non-null    int64
dtypes: float64(4), int64(1), object(3)
memory usage: 31.4+ KB
```

```
sns.countplot(x="Performance_Level",data=df)
plt.title("Performance Distribution")
plt.show()
```




```
df=pd.get_dummies(df,drop_first=True)
df.head()
```

	Study_Hours	Attendance_Percentage	Assignment_Score	Internal_Marks	Performance_Level	Participation_I
0	22.48	84.26	86.79	76.68	1	Fa
1	19.31	94.09	81.10	56.73	1	T
2	23.24	61.01	70.72	52.73	0	Fa
3	27.62	80.63	62.24	64.95	0	T
4	18.83	68.49	78.38	62.45	0	Fa

Next steps: Generate code with df New interactive sheet

```
X=df.drop("Performance_Level",axis=1)
y=df["Performance_Level"]
```

```
sc=StandardScaler()
X=sc.fit_transform(X)
```

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=0)
```

```
models={
    "Random Forest":RandomForestClassifier(),
    "Decision Tree":DecisionTreeClassifier(),
    "Logistic Regression":LogisticRegression(max_iter=2000),
    "KNN":KNeighborsClassifier(),
    "XGBoost":XGBClassifier(eval_metric="mlogloss")
}
```

```
results=[]
for name,model in models.items():
    model.fit(X_train,y_train)
    pred=model.predict(X_test)
    acc=accuracy_score(y_test,pred)
    pre=precision_score(y_test,pred,average="weighted")
    rec=recall_score(y_test,pred,average="weighted")
    f1w=f1_score(y_test,pred,average="weighted")
    f1m=f1_score(y_test,pred,average="macro")
    f1mi=f1_score(y_test,pred,average="micro")
```

```
    f1mi=f1_score(y_test,pred,average="micro")
    results.append([name,acc,pre,rec,f1w,f1m,f1mi])
results_df=pd.DataFrame(results,columns=[
    "Model","Accuracy","Precision","Recall",
    "F1_weighted","F1_macro","F1_micro"])
results_df
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

1 to 5 of 5 entries Filter

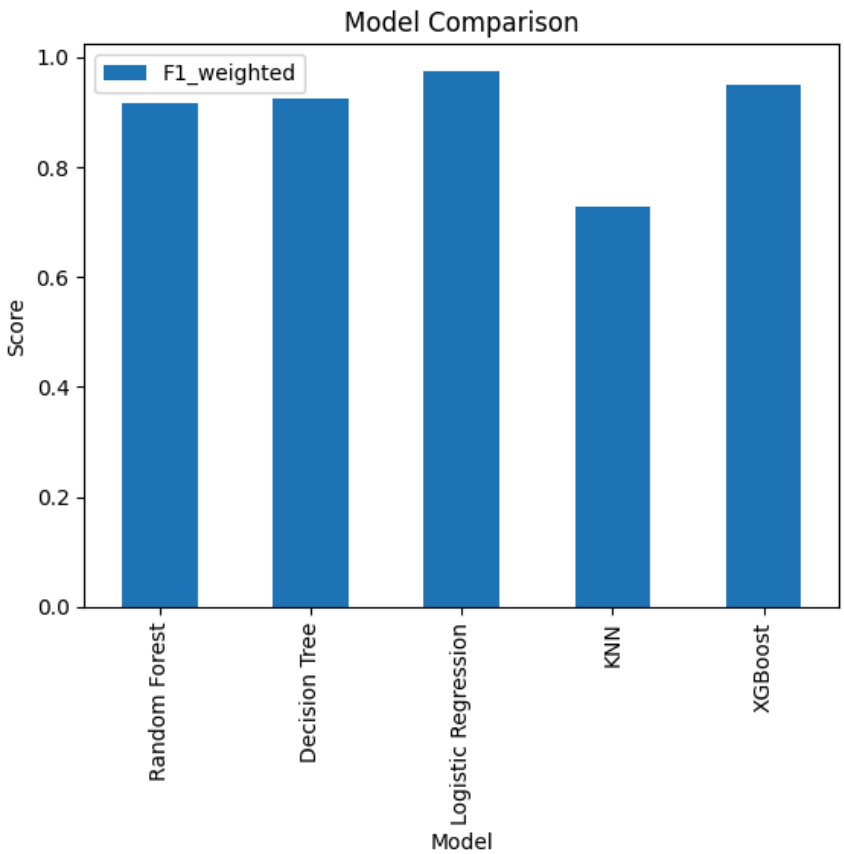
index	Model	Accuracy	Precision	Recall	F1_weighted	F1_macro	F1_micro
0	Random Forest	0.93	0.9263665466186474	0.93	0.9253210077204389	0.6231206826493295	0.93
1	Decision Tree	0.93	0.9231232436772382	0.93	0.9253700305810397	0.6230377166156983	0.93
2	Logistic Regression	0.98	0.9702272727272728	0.98	0.9750574712643677	0.6568828680897646	0.98
3	KNN	0.73	0.736728221597752	0.73	0.7267455501618123	0.48901024811218985	0.73
4	XGBoost	0.95	0.9520426579163248	0.95	0.9501546232347181	0.9659828883620246	0.95

Show 25 per page

Like what you see? Visit the data table notebook to learn more about interactive tables.

Next steps: Generate code with `results_df` New interactive sheet

```
results_df.plot(x="Model",y="F1_weighted",kind="bar")
plt.title("Model Comparison")
plt.ylabel("Score")
plt.show()
```

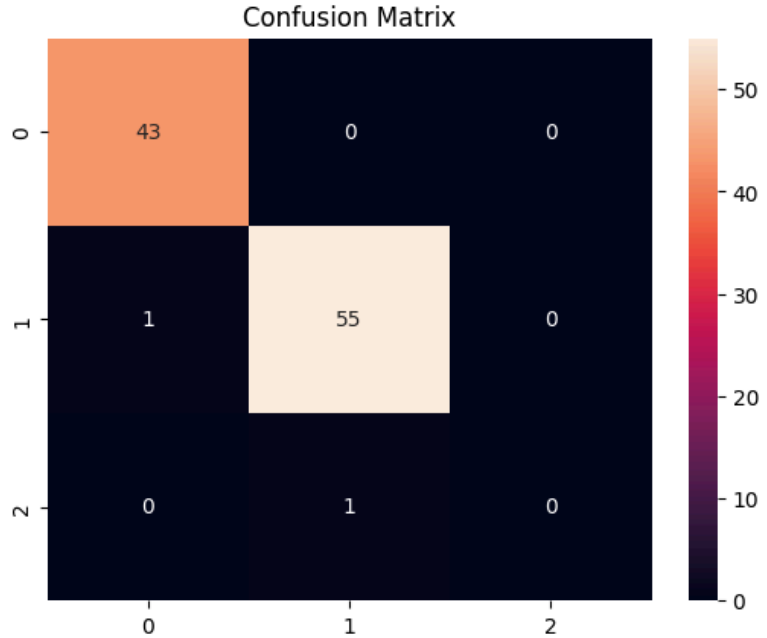



```
best_name=results_df.sort_values("F1_weighted",ascending=False).iloc[0,0]
best_model=models[best_name]
print("Best Model:",best_name)
```

```
Best Model: Logistic Regression
```

```
with(open("best_model","wb")) as f:
  pickle.dump(best_model,f)
```

```
pred=best_model.predict(X_test)
cm=confusion_matrix(y_test,pred)
sns.heatmap(cm,annot=True,fmt='d')
plt.title("Confusion Matrix")
plt.show()
```




```
print(classification_report(y_test,pred))
```

```
              precision    recall  f1-score   support

           0       0.98      1.00      0.99        43
           1       0.98      0.98      0.98        56
           2       0.00      0.00      0.00         1

    accuracy                           0.98       100
   macro avg       0.65      0.66      0.66       100
weighted avg       0.97      0.98      0.98       100

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: P
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```